សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

ROYAL UNIVERSITY OF PHNOM PENH

ប្រធានបទនិក្ខេបបទ

Modernizing Cambodia's Tax Data Warehouse:

A Data Vault 2.0 and Medallion Architecture Approach

A Thesis

In Partial Fulfilment of the Requirement for the Degree of

Master of Science in Data Science and Engineering

Mr. Sot So

March 22th, 2026

សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

ROYAL UNIVERSITY OF PHNOM PENH

ប្រធានបទនិក្ខេបបទ

Modernizing Cambodia's Tax Data Warehouse:

A Data Vault 2.0 and Medallion Architecture Approach

A Thesis

In Partial Fulfilment of the Requirement for the Degree of

Master of Science in Data Science and Engineering

Mr. Sot So

Examination committee: Dr. Khim Chamroeun

Dr. Mao Makara

Mr. Chhim Bunchhun

Mr. Chap Chan Piseth

March 22th, 2026

ABSTRACT IN KHMER

អគ្គនាយកដ្ឋានពន្ធដារនៃប្រទេសកម្ពុជាកំពុងប្រឈមមុខនឹងសម្ពាធក្នុងការធ្វើទំនើបកម្មហេដ្ឋារចនាសម្ព័ន្ធទិន្នន័យ ខណៈពេលដែលរដ្ឋបាលសារពើពន្ធមានការកើនឡើងនូវប្រតិបត្តិការ ភាពស្មុគស្មាញនៃបទប្បញ្ញត្តិ និងតម្រូវការរបាយការណ៍។ការអភិវឌ្ឍន៍ឃ្លាំងទិន្នន័យ(Data Warehouse) ជាលក្ខណៈប្រពៃណី ជាពិសេសគឺ Kimball មានការលំបាកក្នុងការសម្របខ្លួនទៅនឹងការផ្លាស់ប្តូ រចនាសម្ព័ន្ធប្រភពទិន្នន័យ ដែលមានការវិវត្ត ការរក្សាទុកប្រវត្តិប្រតិបត្តិការឱ្យបានពេញលេញ(Full Historical Records) និងការគាំទ្រដល់ Agile Development Cycles។ ចំណុចខ្សោយទាំងនេះដះឥទ្ធិពលធ្ងន់ធ្ងរជាពិសេសនៅក្នុងបរិបទរដ្ឋាភិបាល ដែលមានការធ្វើសវនកម្ម និងការអនុលោមតាមបទប្បញ្ញត្តិ គឺជារឿងសំខាន់បំផុត។ ការសិក្សាស្រាវជ្រាវនេះបានដោះស្រាយបញ្ហាប្រឈម ខាងលើតាមរយៈការរចនាការអនុវត្ត និងការអភិវឌ្ឍន៍ឃ្លាំងទិន្នន័យតាមវិធីសាស្ត្រ **Data Vault 2.0** ស្របតាមទម្រង់ **Medallion Architecture** សម្រាប់ប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យរបស់ពន្ធដារកម្ពុជាដែលត្រូវបានរៀបចំឡើង ដោយប្រើប្រាស់ Microsoft Visual Studio 2026 និង SSIS 2022 នៅលើ Microsoft SQL Server 2025។

ការអនុវត្តនេះប្រើប្រាស់រចនាសម្ព័ន Hub Satellite និង Link របស់វិធីសាស្ត្រ Data Vault 2.0 ជាមូលដ្ឋាននៅក្នុងទម្រង់ Medallion Architecture ដែលមាន ៤ ស្រទាប់៖ ស្រទាប់ Staging (Tables ៩), ស្រទាប់ Bronze (Hubs ៩, Satellites ៩, Links ៥), ស្រទាប់ Silver (Point-in-Time ៣, Bridge ១, Business Vault ២), និងស្រទាប់ Gold (Dimensions ៧, Fact ៤)។ ETL Pipeline ត្រូវបានរៀបចំក្នុងរចនាសម្ព័ន្ធ SSIS តាមហានានុក្រមដែលរួមមាន Master Packages ៣, Orchestrator Packages ៨, និង Child Packages ៤៩ (សរុប ៦០) ដោយផ្តល់នូវ Batch-Step Logging, Watermark-Based Incremental Loading, និង Error Handling។ ជាមួយ Simulated Tax Database ដែលមានអ្នកជាប់ពន្ធ ការប្រកាសពន្ធ ការបង់ប្រាក់ និងមន្ត្រីពន្ធដារត្រូវបានប្រើជា Source System នៅលើមូលដ្ឋានទិន្នន័យចំនួន ៦។

ការវាយតម្លៃ Proof-Of-Concept លើបញ្ហាប្រឈមដែលបានកំណត់ទាំង ៤ ខាងលើគឺ Schema Rigidity, Historical Tracking, Scalable ETL ជាមួយ Error Handling, និង Business Rules Derivation បានផ្តល់នូវភស្តុតាងជាក់ស្តែងថា ការអនុវត្តនិងការអភិវឌ្ឍន៍ឃ្លាំងទិន្នន័យតាមវិធីសាស្ត្រ Data Vault 2.0 និងទម្រង់ Medallion Architecture នេះអាចដោះស្រាយរាល់បញ្ហាប្រឈមនីមួយៗ ជាមួយនឹងការធ្វើឱ្យប្រសើរឡើង ដែលអាចវាស់វែងបានបើប្រៀបធៀបទៅនឹងវិធីសាស្ត្រប្រពៃណី។ ការបំបែក Hub និង Satellite របស់វិធីសាស្ត្រ Data Vault 2.0

អនុញ្ញាតឱ្យមាន Schema Evolution ដោយគ្មានផលប៉ះពាល់(Zero-Impact), ទម្រង់នៃការ Insert-Only Satellite រក្សាទុកនូវប្រវត្តិការយ៉ាងពេញលេញក្នុងការគាំទ្រដល់សវនកម្ម, ក្របខ័ណ្ឌគ្រប់គ្រង ETL Pipeline ដែលផ្តល់នូវ Step-Level Error Recovery ជាមួយនឹងការកត់ត្រា Batch Logging យ៉ាងទូលំទូលាយ។ លើសពីនេះ Pipeline កម្រិត Silver និង Gold បំប្លែងទិន្នន័យពៅ Bronze(Raw Vault) ទៅជាវង្វាស់វិភាគ Silver(Business Vault) ដែលអាចប្រើប្រាស់បានសម្រាប់អាជីវកម្ម។ លទ្ធផលនៃការរកឃើញទាំងនេះ ដែលមានការគាំទ្រ ពីការសិក្សាស្រាវជ្រាវដែលមានស្រាប់រវាងវិធីសាស្ត្រ Data Vault 2.0 នៅក្នុងឯកសារស្រាវជ្រាវ បានផ្តល់នូវ Replicable Reference Architecture សម្រាប់ការធ្វើទំនើបកម្មប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យពន្ធដារនៅកម្ពុជា និងប្រទេសកំពុងអភិវឌ្ឍន៍ដទៃទៀតដែលមានលក្ខណៈសេដ្ឋកិច្ចប្រហាក់ប្រហែល។

ABSTRACT

Cambodia's General Department of Taxation faces mounting pressure to modernize its data infrastructure as tax administration grows in volume, regulatory complexity, and reporting requirements. Traditional data warehouse approaches, notably the Kimball star schema, struggle to accommodate evolving source structures, preserve complete historical records, and support agile development cycles, which are particularly acute in government contexts where auditability and regulatory compliance are essential. The present study addresses these challenges by designing, implementing, and evaluating a Data Vault 2.0 data warehouse for the Cambodia tax system, organized within a medallion architecture by using Microsoft Visual Studio 2026 and SSIS 2022 on Microsoft SQL Server 2025.

The implementation employs Data Vault 2.0's hub-satellite-link pattern within a four-layer medallion architecture: Staging (9 source tables), Bronze (9 Hubs, 9 Satellites, 5 Links), Silver (3 Point-in-Time tables, 1 Bridge, 2 Business Vault tables), and Gold (7 Dimensions, 4 Fact tables in a star schema). The ETL pipeline is orchestrated through a hierarchical SSIS architecture comprising 3 master packages, 8 orchestrator packages, and 49 child packages (60 total). These packages provide batch-and-step logging, watermark-based incremental loading, and error handling. A simulated tax database with taxpayers, declarations, payments, and officer assignments is the source system across six databases.

The proof-of-concept evaluation across four identified challenges (schema rigidity, historical tracking, scalable ETL with error handling, and business rules derivation) provides concrete evidence that the implemented architecture addresses each challenge through specific design patterns. Data Vault 2.0's hub-satellite separation enables zero-impact schema evolution, its insert-only satellite pattern preserves complete historical audit trails, the ETL control framework provides step-level error recovery with full batch logging, and the Silver-Gold layer pipeline transforms raw vault data into business-ready analytical metrics. These findings, supported by existing studies of Data Vault 2.0 methodology with Medallion Architecture approaches in the literature, offer a replicable reference architecture for modernizing tax data systems in Cambodia and comparable developing economies.

Keywords: Data Vault 2.0, Medallion Architecture, Data Warehouse, ETL Pipeline, Cambodia Tax System, SQL Server, SSIS, Star Schema, Incremental Loading, Government Digital Transformation

SUPERVISOR'S RESEARCH SUPERVISION STATEMENT

TO WHOM IT MAY CONCERN

Name of program: **Master of Science in Data Science and Engineering**

Name of candidate: **Mr. Sot So**

Title of research report: Modernizing Cambodia's Tax Data Warehouse: A Data Vault 2.0 and Medallion Architecture Approach

This is to certify that the research carried out for the above titled Master's research report was completed by the above-named candidate under my direct supervision. This thesis material has not been used for any other degree. I played the following part in the preparation of this research report:

Supervisor's name : .....

Supervisor's signature: .....

Date : .....

CANDIDATE'S STATEMENT

TO WHOM IT MAY CONCERN

This is to certify that the research report entitled "**Modernizing Cambodia's Tax Data Warehouse: A Data Vault 2.0 and Medallion Architecture Approach**" which I, Mr. Sot So, am submitting for my Master of Science degree at Royal University of Phnom Penh, Faculty of Engineering, is my own work and has not been used for any other degree or course at this or any other school.

The work submitted for the degree of **Master of Science** at the **Royal University of Phnom Penh** is entirely my own and, furthermore, has not been used to fulfill the requirements of any other qualification in whole or in part, at this or any other University or equivalent institution.

No reference to, or quotation from, this document may be made without the written approval of the author.

Signed by (the candidate) : .....

Date : .....

Supervisor's signature : .....

Date : .....

ACKNOWLEDGEMENT

I wish to express my deepest gratitude to my wife, **Mrs. Om Sreypov**, and my four children, **So Monireach**, **So Monitepy**, **So Monineath**, and **So Monin**, as well as my beloved parents. Their unwavering encouragement and moral support have been the cornerstone of my academic journey. My wife's devotion (her time, energy, patience, and financial support) made it possible to pursue this degree while fulfilling professional responsibilities. The guidance and values instilled by my parents shaped the perseverance that carried this work to completion.

I am profoundly indebted to my research supervisor, **Mr. Chap Chanpiseth**, for his invaluable guidance, scholarly mentorship, and commitment to academic excellence. His constructive feedback sharpened the rigour of this research, his insightful advice helped navigate complex design decisions, and his generous investment of time was instrumental in elevating the quality of every chapter.

My sincere appreciation is extended to **Mr. Ban Dy Vitou**, Director of the Department of Information Communication Technology at the **General Department of Taxation**, whose leadership, motivation, and institutional support created an environment in which professional growth and scholarly achievement could coexist. His confidence in my abilities has been a constant source of strength and inspiration.

I am grateful to **Dr. Srun Sovila,** Program Director and the faculty of the **Master of Science in Data Science and Engineering Program** at the **Royal University of Phnom Penh**, whose rigorous teaching and genuine mentorship broadened my academic perspective and deepened my command of the subject matter. Their dedication to nurturing data professionals in Cambodia has encouraged me to approach every problem with analytical discipline.

Finally, I wish to acknowledge **Mr. Chen Phirum** and **Mr. Soun Pisey**, together with my colleagues and fellow students. Their encouragement and moral support provided strength during the most demanding periods of this journey.

To all who contributed to this journey, I extend my sincerest appreciation. This accomplishment would not have been possible without you.

TABLE OF CONTENTS

Page

Abstract in Khmer (i)

Abstract in English ..... (ii)

Supervisor's Research Supervision Statement ..... (iii)

Candidate's Statement ..... (iv)

Acknowledgement ..... (v)

Table of Contents ..... (vi)

List of Tables ..... (vii)

List of Figures ..... (viii)

CHAPTER 1 INTRODUCTION ..... (1)

1.1 Background to the Study ..... (1)

1.2 Problem Statement ..... (2)

1.3 Aim and Objectives of the Study ..... (3)

1.4 Research Gap and Contributions ..... (4)

1.5 Limitation and Scope ..... (5)

CHAPTER 2 LITERATURE REVIEW ..... (6)

2.1 Data Vault 2.0 Methodology ..... (6)

2.2 Medallion Architecture for Data Warehousing ..... (8)

2.3 ETL Frameworks and Best Practices ..... (10)

2.4 Related Work ..... (12)

2.5 Summary and Conceptual Framework ..... (14)

CHAPTER 3 METHODOLOGY ..... (15)

3.1 System Architecture Overview ..... (15)

3.2 Source Database Design ..... (16)

3.3 ETL Control Framework ..... (17)
3.4 Data Warehouse Layer Design ..... (18)
3.5 ETL Pipeline Design ..... (20)
3.6 Technology Stack ..... (22)

CHAPTER 4 IMPLEMENTATION ..... (23)
4.1 Staging Layer Implementation ..... (23)
4.2 Bronze Layer Implementation (Data Vault) ..... (25)
4.3 Silver Layer Implementation (Business Vault) ..... (28)
4.4 Gold Layer Implementation (Information Vault) ..... (30)
4.5 Master Orchestration ..... (32)

CHAPTER 5 RESULTS AND DISCUSSION ..... (34)
5.1 Results ..... (34)
5.2 Discussion ..... (36)
5.2.1 Challenge 1: Schema Rigidity ..... (36)
5.2.2 Challenge 2: Historical Tracking ..... (37)
5.2.3 Challenge 3: Scalable ETL with Error Handling ..... (38)
5.2.4 Challenge 4: Business Rules Gap ..... (39)
5.3 Threats to Validity ..... (46)

CHAPTER 6 CONCLUSION AND FUTURE WORK ..... (48)

REFERENCES ..... (50)
APPENDICES ..... (53)
Appendix A: SQL Script Files ..... (53)
Appendix B: SSIS Package Inventory ..... (55)

CHAPTER 1

INTRODUCTION

1.1 Background to the Study

Cambodia's tax administration system has been undergoing significant modernization in recent years. The General Department of Taxation (GDT) has been implementing digital systems to improve tax collection efficiency, compliance monitoring, and revenue forecasting. With thousands of taxpayers filing monthly and annual declarations, the volume of transactional data generated by these systems continues to grow rapidly. Financial institutions, government agencies, and regulatory bodies increasingly rely on historical tax data to make informed decisions about economic policy, compliance enforcement, and resource allocation.

Data warehousing provides the foundation for transforming raw transactional data into structured analytical information. Traditional approaches such as the **Kimball** dimensional model and the **Inmon** corporate information factory have been widely adopted across industries. However, these approaches face challenges when applied to government tax systems such as Cambodia's where regulatory requirements change frequently, source systems evolve over time, and complete audit trails must be maintained for legal and compliance purposes.

Data Vault 2.0, introduced by **Dan Linstedt**, is a modern data warehousing methodology specifically designed to address these challenges. It separates business keys (Hubs), descriptive attributes (Satellites), and relationships (Links) into distinct table types. This separation enables parallel development, full historical tracking, and flexible adaptation to source system changes. When combined with the medallion architecture pattern, which organizes data processing into distinct layers (Staging, Bronze, Silver, Gold), Data Vault 2.0 provides a complete framework for enterprise-scale data warehousing.

The present work implements a complete Data Vault 2.0 data warehouse for the Cambodia tax system on Microsoft SQL Server 2025 using Microsoft Visual Studio 2026 and SSIS 2022. The Data Vault 2.0 methodology is the core modeling approach. It provides

Hubs, Satellites, and Links for flexible, auditable data storage. The medallion architecture (Staging, Bronze, Silver, Gold) provides the structural layering that organizes the ETL Pipeline from raw ingestion through business transformation to analytical delivery.

1.2 Problem Statement

Traditional data warehouse designs face several critical limitations when applied to tax administration systems. First, star schema models require significant restructuring when source systems change, which happens frequently in government IT environments as regulations evolve. When a new tax type is introduced or a reporting requirement changes, the entire dimensional model may need to be redesigned, causing costly downtime and development effort.

Second, traditional approaches often struggle to maintain complete historical records. In tax administration, regulatory bodies require full audit trails showing exactly what data was recorded, when it changed, and what the previous values were. Standard SCD Type 1 overwrites previous values entirely, while SCD Type 2 preserves history at the dimensional layer but requires upfront design decisions about which attributes to track. Data Vault 2.0 captures all attribute changes automatically at the raw vault layer through insert-only Satellites. This provides a complete audit trail independent of downstream dimensional design choices.

Third, the growing volume of tax data demands scalable ETL (Extract, Transform, Load) processes that can handle both full and incremental data loads efficiently. Many existing implementations rely on manual processes or simple scripts that lack proper error handling, logging, and recovery mechanisms. When ETL processes fail partway through execution, there is often no systematic way to identify what succeeded, what failed, and how to recover.

Fourth, there is a gap between raw operational data and business-ready analytical information [1,8]. Tax administrators need derived metrics such as compliance scores, payment behavior patterns, and monthly revenue trends that require complex business logic applied consistently across the data warehouse. Without a structured approach to implementing these business rules, analytical outputs may be inconsistent or unreliable.

These problems highlight the need for a modern data warehouse architecture that can handle evolving source systems, maintain complete audit trails, support scalable ETL processes, and deliver reliable business intelligence for tax administration.

1.3 Aim and Objectives of the Study

The study aims to design and implement a complete Data Vault 2.0 data warehouse for the Cambodia tax system using medallion architecture. The study demonstrates how this modern approach addresses the limitations of traditional data warehousing in government tax administration.

The specific objectives are:

1. To design a source database that simulates a realistic tax administration system with taxpayers, declarations, payments, officers, and lookup tables.
2. To implement a four-layer medallion architecture (Staging, Bronze, Silver, Gold) following Data Vault 2.0 methodology on Microsoft SQL Server 2025.
3. To develop a complete ETL pipeline with SSIS packages supporting both full and incremental loading with watermark-based change detection.
4. To implement an ETL control framework with batch logging, step tracking, error handling, and audit capabilities.
5. To create business vault constructs (Point-in-Time tables, Bridge tables, Business tables) that derive analytical metrics from raw vault data.
6. To produce a gold layer information vault (star schema) suitable for business intelligence reporting and dashboard consumption.
7. To orchestrate the complete pipeline through SSIS master packages that execute all layers in the correct dependency order.

1.4 Research Gap and Contributions

Despite growing industry adoption of Data Vault 2.0, the academic literature remains limited in several respects. Yessad and Labiod [9] present a comparative theoretical study of Inmon, Kimball, and Data Vault approaches but do not provide a complete end-to-end implementation or reproducible proof-of-concept evaluation. Naamane and Jovanovic [10] conduct an empirical comparison in the telecommunications domain, providing evidence of Data Vault's auditability advantages, but their work does not address government tax administration where regulatory compliance and audit trail requirements present distinct challenges. Vines [11] evaluates Data Vault versus dimensional modeling using the TPC-DS benchmark dataset, focusing on load speed and

storage efficiency, but does not address the complete medallion architecture pipeline or government domain applications. A recent comparative analysis by Singh [12] examines architectural resilience across healthcare and financial contexts but relies on theoretical frameworks rather than hands-on implementation evidence.

In addition, most existing studies focus on individual aspects (modeling patterns, ETL performance, or schema evolution) rather than presenting a complete end-to-end implementation spanning all four medallion layers with integrated ETL control frameworks. The application of Data Vault 2.0 in government tax administration, where auditability and regulatory compliance are critical, has received particularly limited attention in peer-reviewed literature.

This research makes the following contributions:

1. A complete, reproducible implementation of a four-layer medallion architecture (Staging, Bronze, Silver, Gold) using Data Vault 2.0 methodology on Microsoft SQL Server 2025, comprising 67 database objects across six databases and 60 SSIS packages (3 master, 8 orchestrators, and 49 child packages). This represents the first published end-to-end DV2 + Medallion implementation for the government tax administration domain, addressing the gap identified across all four reviewed studies [9-12].
2. An integrated ETL control framework with batch logging, step tracking, watermark-based incremental loading, and error handling that provides production-grade pipeline observability. The framework demonstrates step-level recovery capabilities and achieves effective incremental change detection with SAT versioning and SCD2 history tracking verified across all layers.
3. A proof-of-concept evaluation demonstrating how Data Vault 2.0 addresses four specific challenges in Cambodia tax system data warehousing: schema rigidity (zero-impact Satellite addition), historical tracking (insert-only audit trail), scalable ETL (step-level error recovery with batch logging), and business rules derivation (compliance scoring via Business Vault). Each challenge validated with before/after evidence through reproducible SQL scripts and SSIS package execution [33].
4. A detailed Technical Implementation Guide document provided as a companion reproducibility artifact detailing the complete SSIS package architecture, connection manager configurations, parameter mappings, execution workflows,

and step-by-step deployment instructions. This guide enables independent researchers and practitioners to reproduce the entire Data Vault 2.0 + Medallion pipeline on their own SQL Server environments, supporting the reproducibility principle essential for academic research validation.

5. A *Source Database Verification* companion document providing evidence-based validation of the source database structure and sample data quality through SSMS-verified results, including table inventory, referential integrity proof, data distribution analysis, and a 12-point verification checklist confirming the database meets all requirements described in Section 3.2 [34].

1.5 Limitation and Scope

1.5.1 Limitations

1. Simulated Data: The source database uses programmatically generated sample data rather than real tax records due to data privacy and confidentiality constraints. The quality and structure of this sample data is verified in the companion Source Database Verification document [34], which confirms 9 tables, 57,990 records, zero orphan records, zero NULLs on critical columns, and realistic distributions across all entities.
2. Single Source System: The implementation loads data from a single source database, whereas production environments typically integrate multiple source systems.
3. No Real-Time Processing: The ETL pipeline operates in batch mode only and does not support real-time or near-real-time data streaming.
4. Performance Scale: The implementation was conducted at proof-of-concept scale (1,000 taxpayers, ~58,000 source records). Production-scale validation with millions of taxpayers would require additional optimizations such as column store indexes, table partitioning, and parallel loading strategies. Quantitative performance benchmarking is identified as future work.
5. Security: Role-based access control, data encryption, and row-level security were not implemented in this prototype.

1.5.2 Scope

1. The project covers the complete data warehouse lifecycle from source database creation through gold layer information vault.
2. Nine source tables are processed through all four layers of the medallion architecture.
3. The Bronze layer implements 9 Hubs, 9 Satellites, and 5 Links following Data Vault 2.0 standards.
4. The Silver layer includes 3 Point-in-Time tables, 1 Bridge table, and 2 Business Vault tables.
5. The Gold layer delivers 7 Dimensions (including SCD Type 2) and 4 Fact tables.
6. SSIS package architecture is designed for both full and incremental load orchestration.

CHAPTER 2

LITERATURE REVIEW

2.1 Data Vault 2.0 Methodology [1]

Data Vault 2.0 is a data warehousing methodology created by Dan Linstedt that provides a system of business intelligence designed to be agile, scalable, and capable of maintaining full historical records. The original Data Vault concept (version 1.0) was first introduced by Linstedt in 2000 as a hybrid approach combining the best aspects of third normal form (3NF) and star schema designs. The methodology evolved significantly over the following decade, culminating in the publication of "Building a Scalable Data Warehouse with Data Vault 2.0" [1] in 2015, which formalized the methodology with additions including hash-based key generation, NoSQL integration patterns, and standardized loading patterns. Hultgren [6] further contributed to the methodology's practical adoption through "Modeling the Agile Data Warehouse with Data Vault," providing detailed guidance on agile development patterns and ensemble modeling techniques. More recently, Vines and Samoila [25] present a detailed overview of the Data Vault methodology and its benefits, confirming its continued relevance and growing industry adoption in modern data engineering contexts. Since its formalization, Data Vault 2.0 has been adopted by organizations worldwide for enterprise data warehousing.

The core modeling pattern of Data Vault 2.0 consists of three entity types: Hubs, Links, and Satellites. Hubs store unique business keys that represent core business concepts such as customers, products, or transactions. Each Hub record is insert-only and contains a hash key, the business key, a load timestamp, and a record source. Links capture the relationships between Hubs, representing how business entities are connected. Like Hubs, Links are insert-only and use hash keys derived from the combined business keys of the related Hubs. Satellites store the descriptive attributes and context associated with Hubs and Links. Satellites use hash difference values to detect changes. This enables efficient incremental loading and full historical tracking.

A key advantage of Data Vault 2.0 over traditional approaches is its support for parallel and agile development. Because Hubs, Links, and Satellites are independent structures, multiple development teams can work simultaneously on different parts of the

data warehouse without conflicts. Also, source system changes only affect the relevant Satellites, not the core Hub and Link structure, minimizing the impact of upstream changes on the data warehouse.

2.1.1 Core and Extended Components

The following table summarizes both the core Data Vault 2.0 components (Hub, Link, Satellite) that form the Raw Vault, and the extended constructs (Point-in-Time, Bridge, Business Vault) that comprise the Business Vault layer [1,6]. The core components are defined by the Data Vault 2.0 standard, while the extended constructs are architectural patterns recommended for simplifying queries and deriving business metrics from the Raw Vault.

Table 2.1: *Core and Extended Components of Data Vault 2.0*

Component	Purpose	Key Characteristics
Hub	Store unique business keys	Insert-only, hash key, business key, load date, record source
Link	Capture relationships between Hubs	Insert-only, hash key, foreign hash keys, load date
Satellite	Store descriptive attributes	Hash diff for change detection, full history, load date
Point-in-Time (PIT)	Snapshot of satellite state at a point in time	Simplifies queries, pre-joined satellite data
Bridge	Pre-joined link traversals	Performance optimization for complex joins
Business Vault	Derived business rules and calculations	Compliance scores, metrics, computed values

2.1.2 Hash Key Generation

Data Vault 2.0 uses hash keys (typically MD5 or SHA-256) as surrogate keys instead of sequence-generated integers. Hash keys provide deterministic key generation, meaning the same business key always produces the same hash key regardless of when or

where the data is loaded. This eliminates the need for key lookups during ETL processing and enables parallel loading across multiple source systems.

2.1.3 Dataset and Application

The Data Vault 2.0 methodology has been applied across industries including banking, insurance, healthcare, telecommunications, and government [1,6,12,21]. Its emphasis on auditability and historical completeness makes it particularly suitable for regulated industries where data lineage and change tracking are mandatory requirements [1,8].

2.2 Medallion Architecture for Data Warehousing [2,26]

The medallion architecture, popularized by Databricks and subsequently adopted by Microsoft as the recommended design pattern for Microsoft Fabric [26], organizes data processing into distinct quality layers. While originally described in the context of data lakehouse architectures, the pattern is equally applicable to traditional relational data warehouses. The architecture defines three to four layers, each serving a specific purpose in the data refinement process.

The Bronze layer (or Raw Vault in Data Vault terminology) contains data loaded directly from source systems with minimal transformation. Data in this layer preserves the original structure and content of the source, serving as the single source of truth for all downstream processing. The Silver layer (Business Vault) applies business rules, data quality validations, and cross-referencing logic to produce cleansed and enriched data. The Gold layer (Information Vault) presents data in consumption-ready formats such as star schemas, optimized for reporting and analytical queries.

Several organizations have documented their adoption of medallion architecture for enterprise data warehousing. Microsoft has adopted the medallion pattern as the recommended design approach for its Fabric platform, implementing it through OneLake lakehouses with bronze, silver, and gold layers that ensure ACID compliance as data progresses through validations and transformations [26]. The architecture provides clear separation of concerns, making it easier to troubleshoot data quality issues by examining each layer independently. It also supports iterative development, allowing teams to deliver value at the Bronze layer while progressively enhancing Silver and Gold layers.

2.2.1 Layer Architecture

Table 2.2: *Medallion Architecture Layer Summary*

Layer	Database	Purpose	Objects
Staging	DV_Staging	Raw extraction from source	One table per source entity
Bronze	DV_Bronze	Data Vault 2.0 (Hubs, Satellites, Links)	Hubs, Satellites, Links
Silver	DV_Silver	Business Vault (PIT, Bridge, Business)	PITs, Bridges, Business tables
Gold	DV_Gold	Information Vault (Star Schema)	Dimensions, Facts (Star Schema)

2.2.2 Comparison with Traditional Approaches

Compared to the Kimball approach, which loads data directly into dimensional models, the medallion architecture preserves raw data fidelity while still delivering business-ready dimensional outputs. Compared to the Inmon approach [7], which uses a normalized enterprise data model, medallion architecture with Data Vault provides greater flexibility for source system changes and supports parallel loading patterns. The combination of Data Vault 2.0 at the Bronze layer with star schema at the Gold layer leverages the strengths of both methodologies.

2.3 ETL Frameworks and Best Practices [3]

ETL (Extract, Transform, Load) processes form the backbone of any data warehouse implementation. The ETL approach validates and transforms data before loading it into the data warehouse, guaranteeing data quality and consistency at each processing stage. This approach is particularly suited to on-premises SQL Server environments and regulated government contexts where data must be verified and cleansed before entering the warehouse [4]. Modern ETL frameworks emphasize observability, idempotency, error handling, and incremental processing as essential capabilities for production-grade data pipelines. Our implementation adopts the ETL pattern using SSIS

2022, which provides visual workflow design, package-level error handling, and native integration with SQL Server 2025's execution engine.

Watermark-based incremental loading is a well-established approach for detecting changed records in source systems [3,19]. This technique stores a high-water mark value (typically the maximum UpdatedDate of processed records) and filters subsequent extractions to only records modified after this watermark. This significantly reduces processing time compared to full loads, especially for large source tables. Watermark-based loading is simple to implement, easy to maintain, and fully auditable, making it the preferred approach for batch-oriented ETL pipelines in regulated government contexts where processing transparency and reproducibility are critical requirements [3,19].

Batch logging and step tracking provide observability into ETL execution. A well-designed ETL control framework records batch start and end times, records processed and inserted, error counts, and execution status for each step within a batch. This information is essential for monitoring pipeline health, diagnosing failures, and supporting audit requirements. Parent-child package patterns enable parameter passing (such as BatchID and LoadType) across the pipeline while maintaining clear separation between processing stages. Hash-based change detection at the Bronze layer compares incoming record hashes against existing satellite records, where changed values trigger new row insertions. This insert-only, hash-based approach eliminates the need for complex upsert logic and ensures pipeline recoverability after partial failures.

2.3.1 ETL Control Components

Table 2.3: *ETL Control Framework Components*

Component	Purpose
ETL_Process	Registry of ETL processes with source/target mappings
ETL_BatchLog	Batch-level execution logging (start, end, status, record counts)
ETL_StepLog	Step-level execution logging within batches
ETL_ErrorLog	Detailed error logging with severity, source, and message (SSIS OnError Event Handler with Script Task)
ETL_Watermark	High-water mark values for incremental loading

2.4 Related Work

This section reviews peer-reviewed studies relevant to Data Vault 2.0 implementations, medallion architecture patterns, and comparative evaluations of data warehouse modeling approaches.

2.4.1 Data Vault Comparative Studies

Yessad and Labiod [9] present a comparative study of Inmon, Kimball, and Data Vault modeling approaches, published at the IEEE International Conference on System Reliability and Science (ICSRS 2016). Their work highlights Data Vault's flexibility for source system changes and methodological agility but is limited to theoretical comparison without experimental performance metrics. **Strengths:** thorough three-way comparison (Inmon, Kimball, DV2); establishes theoretical foundation for DV2 adaptability advantages; highlights flexibility for source system changes **Weaknesses:** no experimental implementation or benchmarks; purely theoretical with no empirical evidence; does not address specific domain applications or real-world deployment scenarios.

Naamane and Jovanovic [10] extend this with an empirical comparison in the telecommunications domain, implementing a Customer Experience Data Mart using both Kimball star schema and Data Vault 2.0 with materialized data marts. Their findings provide evidence that Data Vault offers superior auditability and ETL patterns but acknowledges insufficient performance for direct reporting without a dimensional presentation layer, a limitation our medallion architecture addresses through the Gold star schema layer. **Strengths:** empirical comparison with real telecom data; implements both Kimball and DV2 approaches; demonstrates DV2 superior auditability and ETL patterns **Weaknesses:** single telecom case study that may not generalize; no quantitative performance benchmarks; acknowledges DV2 insufficient for direct reporting without presentation layer.

Vines [11] presents the most recent performance-focused study, published at the 23rd International Conference on Informatics in Economy (Springer, 2024). Using the TPC-DS benchmark dataset, the study compares Data Vault and dimensional modeling on data load speed, storage utilization, query execution time, and scalability metrics. The findings demonstrate trade-offs between modeling complexity and performance that our research extends to the Cambodia government tax context with a complete four-layer

architecture. **Strengths:** most recent performance-focused study (2024); uses standardized TPC-DS benchmark dataset; provides quantitative metrics on load speed, storage, query time, and scalability **Weaknesses:** synthetic data only with no real domain application; does not address complete medallion architecture pipeline; no government or compliance context consideration.

Singh [12] provides a comparative analysis of Data Vault 2.0, dimensional modeling, and hybrid frameworks in high-volume environments, evaluating architectural resilience using theoretical performance metrics and security protocols. Their analysis indicates that dimensional modeling retains query performance superiority for end-user reporting while Data Vault 2.0 demonstrates superior resilience in data ingestion, auditability, and schema drift handling — consistent with our empirical findings. **Strengths:** evaluates DV2 architectural resilience across multiple contexts (healthcare, finance); covers security protocols and hybrid frameworks; broad scope **Weaknesses:** theoretical framework with no hands-on implementation; no empirical benchmarks or proof-of-concept evidence; does not test schema evolution in practice.

Table 2.4: *Summary of Strengths and Weaknesses in Existing DV2 Studies*

Study	Key Strengths	Key Weaknesses	Gap Addressed
Yessad & Labiod (2016) IEEE ICSRS	Thorough 3-way comparison; theoretical DV2 adaptability foundation	No implementation; purely theoretical; no domain-specific application	Complete implementation with POC evaluation evidence
Naamane & Jovanovic (2016) IJCSI	Empirical telecom data; implements both approaches; DV2 auditability evidence	Single case study; no performance metrics; DV2 insufficient for direct reporting	Gold layer resolves reporting gap; multi-domain generalizability
Vines (2024) Springer IE Conf.	Recent study; TPC-DS benchmark; quantitative load/storage/query metrics	Synthetic data only; no medallion architecture; no government domain	Cambodia tax domain with 4-layer medallion + compliance context
Singh (2025) IJARSCT	Resilience analysis (healthcare/finance); security evaluation; hybrid frameworks	Theoretical only; no implementation or POC; no empirical benchmarks	Full POC with implementation evidence; DV2 demonstrated on Cambodia tax domain

2.4.2 Data Warehouse Evolution and Schema Adaptation

The challenge of schema evolution in data warehouses has been extensively studied. Bellahsene [13] addresses schema evolution and maintenance in data warehousing systems when underlying data sources change, presenting approaches for dynamically adapting warehouse views. Arora and Gosain [14] survey schema evolution techniques including schema versioning, temporal warehousing, and view maintenance, providing a comparative analysis of methodologies for handling structural changes. Fan and Poulovassilis [23] propose a schema transformation-based approach to schema evolution in data warehousing environments, defining a formal framework of primitive and composite transformations that can be applied to evolving warehouse schemas. Oueslati and Akaichi [24] complement this with a broad survey of data warehouse evolution research, categorizing approaches into schema evolution, data evolution, and ETL evolution, and identifying open challenges in maintaining semantic consistency across schema versions. Subotic [15] proposes a meta-Data Vault model integrating the data warehouse with master data management systems to address schema evolution challenges, tracking data origins, history of changes, and preventing data loss. These studies collectively establish that schema evolution remains a significant challenge in traditional data warehousing, motivating our adoption of the Data Vault 2.0 modeling pattern which inherently accommodates source system changes through its hub-satellite separation.

2.4.3 Medallion and Lakehouse Architectures

Building on the architectural foundation described in Section 2.2, the medallion pattern has attracted significant attention in the academic literature on data engineering evolution. Nambiar and Mundra [16] provide an overview of data warehouse and data lake in modern enterprise data management, examining how layered architectures bridge the gap between raw data storage and analytical consumption. Dhaouadi et al. [17] present a detailed comparison of data warehousing process modeling approaches from classical to contemporary trends, situating the medallion pattern within the broader evolution of data engineering practices. Ponnusamy [18] traces the evolution of enterprise data warehousing from Management Information Systems through data marts to modern cloud-based processing, noting that medallion architecture combines data vault modeling principles with cloud-native data engineering patterns.

2.4.4 Data Management Challenges in Tax Administration

Government tax administrations worldwide face growing data management challenges as digital transformation accelerates. Gandomi and Haider [28] establish a foundational framework for understanding these challenges through the five V's of Big Data — Volume, Velocity, Variety, Veracity, and Value — noting that the transition from traditional data processing to analytics-driven decision making requires reliable architectural approaches. Oussous et al. [29] extend this analysis through a broad survey of Big Data technologies, examining how modern data pipeline architectures address the challenges of scale and complexity in enterprise environments.

In the specific context of tax administration, Shukla et al. [30] demonstrate the application of Big Data analytics for tax fraud detection in the government of Telangana, India, establishing that effective tax compliance analytics requires a well-structured data warehouse capable of integrating data from multiple sources and supporting complex analytical queries. The Inter-American Center of Tax Administrations (CIAT) [31] documents global experiences of Big Data adoption in tax agencies, including the US IRS, the UK HMRC, and Spain's AEAT systems. As Cambodia's General Department of Taxation (GDT) continues its digital transformation, the ability to manage growing data volumes through enterprise data warehouse architectures becomes a critical capability. While the current research operates at proof-of-concept scale, the Data Vault 2.0 methodology's insert-only loading pattern, hash-based key generation, and hub-satellite separation provide an architectural foundation that can accommodate future growth toward larger data volumes [1,16,17], and the medallion architecture's layered approach provides the structured transformation pipeline needed for converting raw data into actionable business intelligence.

2.4.5 ETL Frameworks and Data Quality

Vassiliadis et al. [19] present a thorough survey of ETL technology covering conceptual and logical modeling of ETL processes, individual stage optimization, and end-to-end pipeline management. Their framework for categorizing ETL research informs our ETL control architecture design. The TPC-DS benchmark [20] provides a standardized approach for measuring decision support workloads, establishing metrics for data load throughput, transformation efficiency, and pipeline cost that inform the design principles of ETL control frameworks.

2.4.6 Government and Tax Domain Applications

While data warehousing has been widely applied in commercial domains, applications in government tax administration are underrepresented in academic literature. Khalid [21] discusses challenges of dimensional modeling in business intelligence systems, noting that government contexts introduce additional complexity through regulatory compliance requirements and frequent policy changes. Leonard [22] presents the design and implementation of an enterprise data warehouse with practical considerations for production deployment. Our research addresses this gap by demonstrating Data Vault 2.0 implementation specifically for Cambodia tax system data warehousing, where auditability, complete historical tracking, and regulatory compliance are mandatory requirements rather than optional features.

2.5 Summary and Conceptual Framework

The literature reviewed in this chapter reveals three converging themes that collectively motivate and inform this research. First, existing comparative studies of Data Vault 2.0 [9-12] demonstrate its theoretical and practical advantages in schema flexibility, auditability, and ETL resilience, but remain limited to either theoretical analysis or single-domain case studies without complete end-to-end implementations. Second, the medallion architecture pattern [2,26], combined with the broader evolution of data warehousing from classical approaches to modern layered architectures [16-18], provides a proven structural framework for organizing data processing from raw ingestion through analytical delivery. Third, the ETL literature [3,4,19,20] establishes essential principles — idempotency, watermark-based change detection, batch logging, and error isolation — that are critical for production-grade data pipeline reliability.

The proposed architecture synthesizes these three streams into a unified conceptual framework: Data Vault 2.0 methodology provides the core modeling approach (Hubs, Satellites, Links) for flexible and auditable data storage; the medallion architecture provides the structural layering (Staging, Bronze, Silver, Gold) that organizes data refinement from raw extraction to analytical consumption; and the ETL control framework provides the operational infrastructure (batch logging, step tracking, watermark management, error handling) that ensures pipeline reliability and observability. The application of this integrated framework to the Cambodia tax administration domain

addresses the gap in government-specific Data Vault 2.0 implementations identified across the reviewed literature, while the inclusion of POC evaluation evidence addresses the empirical evidence gap noted in theoretical studies [9,12]. The following table positions this research relative to the key studies reviewed.

Table 2.5: *Research Gap Comparison Matrix*

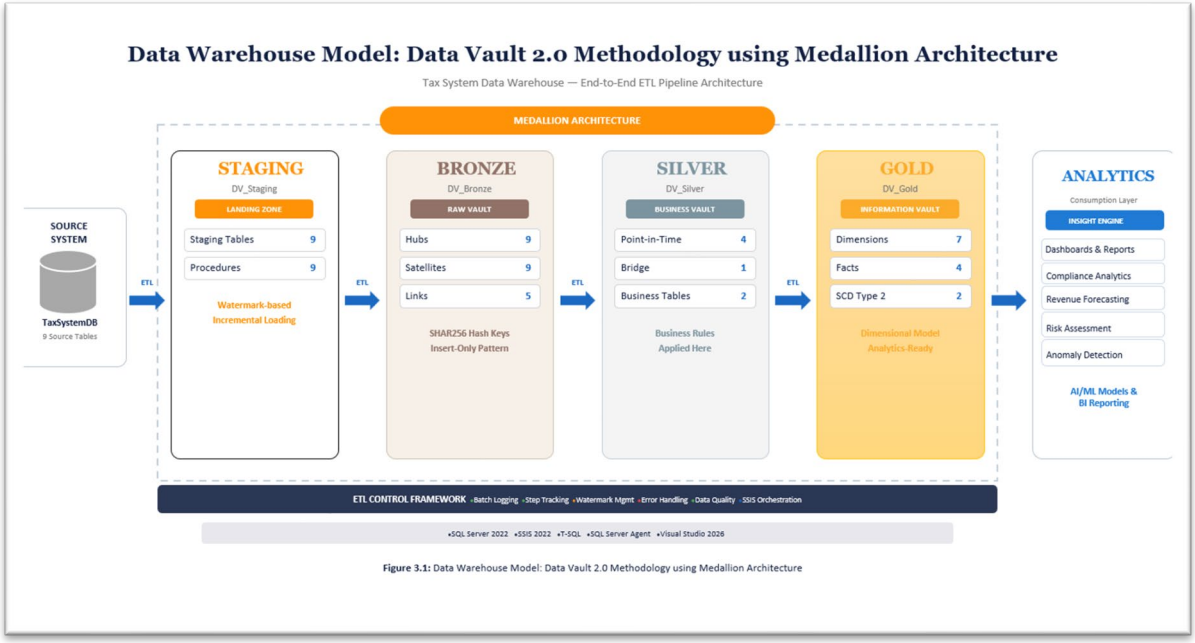
Study	Year	Focus	Limitation Addressed by This Research
Yessad & Labiod [9]	2016	Inmon/Kimball/DV comparison	Theoretical only; no performance benchmarks
Naamane & Jovanovic [10]	2016	DV vs Kimball (telecom)	Single case, no medallion architecture
Vines [11]	2024	DV vs DM (TPC-DS)	No domain-specific implementation
Singh [12]	2025	DV2.0 resilience analysis	Theoretical framework, no hands-on
Subotic [15]	2015	DW schema evolution	No DV2.0 complete implementation
This research	2026	DV2.0 + medallion (Cambodia tax)	End-to-end with POC evidence

CHAPTER 3

METHODOLOGY

3.1 System Architecture Overview

Figure 3.1: Data Warehouse Model — Data Vault 2.0 Methodology using Medallion Architecture



The system implements a Data Vault 2.0 data warehouse methodology, organized through a four-layer medallion architecture with a centralized ETL control framework. Data Vault 2.0 provides the core modeling pattern (Hubs, Satellites, and Links) for storing raw data with full auditability, while the medallion architecture provides the structural layering that separates data ingestion (Staging), raw vault storage (Bronze), business rule application (Silver), and analytical delivery (Gold). Each layer is implemented as a separate SQL Server database. The ETL_Control database manages batch execution, step logging, watermark tracking, and data quality monitoring across all layers.

The complete architecture consists of six databases:

Table 3.1: *Database Architecture Overview*

Database	Layer	Purpose
TaxSystemDB	Source	Cambodia operational tax system with 9 tables (3 lookup, 3 reference, 3 transaction)
DV_Staging	Staging	Temporary landing area for extracted source data
DV_Bronze	Bronze	Data Vault 2.0 raw vault (Hubs, Satellites, Links)
DV_Silver	Silver	Business Vault (PIT tables, Bridge tables, Business metrics)
DV_Gold	Gold	Dimensional model (Star schema for BI reporting)
ETL_Control	Control	ETL orchestration, logging, watermarks, configuration

The ETL pipeline is executed through a hierarchical SSIS package design with Master_Complete_Pipeline.dtsx as the entry point, branching to Master_Full_Load.dtsx or Master_Incremental_Load.dtsx based on watermark state. Each master package orchestrates layer-specific child packages in dependency order, supporting both full load and incremental load modes.

3.2 Source Database Design

The source database (TaxSystemDB) simulates a realistic Cambodia tax administration system. It contains three lookup tables (Category, Structure, Activity), three reference tables (Taxpayer, Owner, Officer), and three transaction tables (MonthlyDeclaration, AnnualDeclaration, Payment).

3.2.1 Source Tables

Table 3.2: *Source Database Tables*

Table	Type	Description	Sample Records

Category	Lookup	Taxpayer categories (Individual, Small Business, Corporation, etc.)	5
Structure	Lookup	Business structures (Sole Proprietorship, Partnership, LLC, etc.)	5
Activity	Lookup	Business activities (Retail, Manufacturing, Services, etc.)	5
Taxpayer	Reference	Registered taxpayers with TaxID, business name, revenue	1,000
Owner	Reference	Business owners with ownership percentages	~2,000
Officer	Reference	Tax collection officers with department assignment	10
MonthlyDeclaration	Transaction	Monthly tax declarations with revenue, tax, penalties	~38,000
AnnualDeclaration	Transaction	Annual tax declarations with yearly summaries	2,785
Payment	Transaction	Tax payments linked to declarations	~15,000

The source database implements data quality constraints including CHECK constraints on numeric fields, foreign key relationships between tables, and filtered indexes for performance. Sample data is generated programmatically using randomized patterns to simulate realistic distributions of taxpayer categories, declaration statuses, payment methods, and temporal patterns spanning four years (2020-2023).

A full verification of the source database structure and sample data quality is documented in the companion *Source Database Verification* document [34], which includes SSMS screenshots confirming: 9 tables with 57,990 records, 10 foreign keys, 19 CHECK constraints, 35 indexes, zero orphan records across 7 relationships, zero NULLs

on 8 critical columns, unique TaxID verification (1,000/1,000), and realistic data distributions across 5 taxpayer categories spanning 2020–2023.

3.3 ETL Control Framework

The ETL Control database provides centralized management of the entire ETL pipeline. It implements a hierarchical logging model where each pipeline execution creates a Batch record, and each processing step within the batch creates a Step record linked to its parent batch.

Key features of the ETL control framework include:

1. Batch Lifecycle: Start/End logging (via Execute SQL Tasks) that track batch execution status, duration, and record counts.
2. Step Tracking: Granular logging of each ETL step with start time, end time, records affected, and error details.
3. Watermark Management: High-water mark values stored per process/table/column for incremental load detection.
4. Data Quality Logging: Schema-level support for recording data quality check results with pass/fail status and violation counts (framework tables defined but not actively exercised by the current POC pipeline).
5. Error Handling: Standardized error capture with SSIS OnError Event Handlers and error message logging at both batch and step levels.

3.4 Data Warehouse Layer Design

Staging Layer (DV_Staging)

The staging layer is the landing zone for the Data Vault 2.0 pipeline, decoupling source system access from downstream processing. It contains 9 tables that mirror the source system structure with additional metadata columns (SourceSystem, RecordSource, DV_LoadDateTime, DV_BatchID) required by the Data Vault 2.0 loading pattern. Each staging SSIS package supports both FULL load (truncate and reload) and INCREMENTAL load (watermark-based delta extraction).

Bronze Layer (DV_Bronze) — Raw Vault (Data Vault 2.0 Core)

The Bronze layer implements the core of the Data Vault 2.0 methodology (the Raw Vault) with 23 objects following the standard Hub-Satellite-Link pattern:

Table 3.3: *Bronze Layer Data Vault Object Summary*

Object Type	Count	Objects
Hubs	9	HUB_Taxpayer, HUB_Declaration, HUB_Payment, HUB_Officer, HUB_Owner, HUB_Category, HUB_Structure, HUB_Activity, HUB_AnnualDecl
Satellites	9	SAT_Taxpayer, SAT_MonthlyDecl, SAT_Payment, SAT_Officer, SAT_Owner, SAT_Category, SAT_Structure, SAT_Activity, SAT_AnnualDecl
Links	5	LNK_TaxpayerDeclaration, LNK_DeclarationPayment, LNK_TaxpayerOfficer, LNK_TaxpayerOwner, LNK_TaxpayerAnnualDecl

All Hub and Satellite tables use SHA-256 hash keys generated from business keys using HASHBYTES('SHA2_256') stored as VARBINARY(32). Satellites use hash difference values computed from concatenated attribute values to detect changes efficiently. Hub and Link tables are insert-only, while Satellites maintain full history through load date and load end date columns.

Silver Layer (DV_Silver) — Business Vault (Data Vault 2.0 Extension)

The Silver layer extends the Data Vault 2.0 architecture through the Business Vault pattern, transforming raw vault data into business-ready constructs:

1. Point-in-Time (PIT) Tables (3): PIT_Taxpayer, PIT_Declaration, PIT_Payment. These provide snapshot views of Hub-Satellite combinations at any given date, simplifying queries that would otherwise require complex temporal joins.
2. Bridge Table (1): BRG_Taxpayer_Owner. Pre-joins the Taxpayer-Owner relationship with ownership details for efficient multi-hop queries.
3. Business Vault Tables (2): BUS_ComplianceScore calculates taxpayer compliance ratings based on declaration timeliness, payment behavior, and penalty history. BUS_MonthlyMetrics aggregates monthly revenue, tax, and payment metrics per taxpayer.

Gold Layer (DV_Gold) — Information Vault (Dimensional Delivery)

The Gold layer is the Information Vault, the consumption-ready output of the Data Vault 2.0 pipeline, presenting data in a star schema optimized for BI reporting:

Table 3.4: *Gold Layer Star Schema Objects*

Type	Count	Objects
Dimensions	7	DIM_Taxpayer (SCD2), DIM_Officer (SCD2), DIM_Category, DIM_Structure, DIM_Activity, DIM_PaymentMethod, DIM_Status
Facts	4	FACT_MonthlyDeclaration, FACT_Payment, FACT_MonthlySnapshot, FACT_DeclarationLifecycle

DIM_Taxpayer and DIM_Officer implement Slowly Changing Dimension Type 2, maintaining full history with effective date ranges and current record flags. Fact tables reference dimension surrogate keys and store measurable values such as tax amounts, payment amounts, and declaration counts.

3.5 ETL Pipeline Design

The ETL pipeline follows a strict dependency order: Staging first loads data from the source, then Bronze loads from Staging, Silver loads from Bronze, and Gold loads from Silver. Each layer's SSIS child packages are called sequentially within the master SSIS package.

3.5.1 Loading Patterns

1. Full Load: Truncates staging tables and reloads all records. Used for initial loads and periodic full refreshes.
2. Incremental Load: Uses watermark values to extract only records modified since the last successful load. As discussed in Section 2.3, watermark-based loading was selected for its simplicity, auditability, and alignment with the batch processing model. Applied at the staging layer; Bronze and higher layers inherently process only new staging records.

3.5.2 Error Handling Strategy

Each SSIS child package includes an OnError Event Handler that captures errors. Errors are logged to both ETL_StepLog (step-level status) and ETL_ErrorLog (detailed error information including ErrorSeverity, ErrorSource, and ErrorMessage columns). A Script Task within the OnError handler captures System::ErrorDescription into a user variable and calls usp_LogError with six parameters (@BatchID, @StepLogID, @ErrorSeverity, @ErrorSource, @ErrorMessage, @ErrorDetails), where @BatchID and @StepLogID default to NULL when unset. This enables precise error categorization and detailed diagnostics. The master orchestration continues processing subsequent steps even if individual steps fail, accumulating an error count for the final batch status. SSIS packages implement OnError event handlers at both child package and orchestrator levels, with error propagation control to ensure sibling packages can continue execution when independent steps fail.

3.6 Technology Stack

Table 3.5: *Technology Stack*

Technology	Version	Purpose
SQL Server	2025 Developer Edition (RTM 17.0)	Database engine, SSIS runtime, SQL Server Agent
SQL Server Integration Services (SSIS)	Included with SQL Server 2025+	ETL package execution and orchestration
T-SQL	-	SSIS packages, Data Flow tasks, ETL logic
SQL Server Agent	Included with SQL Server 2025	Job scheduling for automated pipeline execution
Visual Studio Community	2026	SSIS package design and development (IDE)

CHAPTER 4

IMPLEMENTATION

4.1 Staging Layer Implementation

4.1.1 Goal

Extract data from the TaxSystemDB source system into the DV_Staging database, supporting both full and incremental loading modes with batch and step logging.

4.1.2 Implementation Details

Nine SSIS child packages were created, one for each source table. Each SSIS child package follows an identical pattern: receive a BatchID parameter via Execute SQL Task, start a step log entry, execute the load logic via Data Flow task (full or incremental), update the watermark, and end the step log entry with the record count. A master SSIS package (STG_Full_Load_All.dtsx/ STG_Incremental_Load_All.dtsx) creates a batch and calls all nine staging child packages in sequence.

4.1.3 SSIS Packages Implemented

Table 4.1: *Staging Layer SSIS Packages*

SSIS Package	Source Table	Load Modes
STG_Category.dtsx	Category	Full / Incremental
STG_Structure.dtsx	Structure	Full / Incremental
STG_Activity.dtsx	Activity	Full / Incremental
STG_Taxpayer.dtsx	Taxpayer	Full / Incremental
STG_Owner.dtsx	Owner	Full / Incremental
STG_Officer.dtsx	Officer	Full / Incremental
STG_MonthlyDeclaration.dtsx	MonthlyDeclaration	Full / Incremental
STG_AnnualDeclaration.dtsx	AnnualDeclaration	Full / Incremental

STG_Payment.dtsx	Payment	Full / Incremental

4.1.4 Key Implementation Features

1. Watermark-based incremental loading using ETL_Watermark table with UpdatedDate column tracking.
2. Full load mode truncates target table before insert for clean state recovery.
3. Each SSIS child package logs records processed, start/end times, and errors via usp_StartStep/usp_EndStep.

4.2 Bronze Layer Implementation (Data Vault)

4.2.1 Goal

Load staging data into Data Vault 2.0 structures following hub-satellite-link patterns with hash key generation and change detection.

4.2.2 Hub Loading

Nine Hub SSIS packages insert new business keys from staging into Hub tables. Each package generates a SHA-256 hash key from the business key using HASHBYTES('SHA2_256'), checks for existence using a LEFT JOIN with NULL check, and inserts only new records. Hub tables are append-only and never updated or deleted.

4.2.3 Satellite Loading

Nine Satellite SSIS packages detect attribute changes using hash difference values. For each staging record, the Data Flow task generates a HashDiff from concatenated attribute values and compares it against the current active Satellite record. Only records with changed attributes are inserted, maintaining a complete change history. The previous active record's LoadEndDateTime is updated to close out that version.

4.2.4 Link Loading

Five Link SSIS packages capture relationships between business entities. Each Link record stores hash keys of the related Hubs along with its own composite hash key. Like Hubs, Links are insert-only and never modified.

4.2.5 SSIS Packages Implemented

Table 4.2: *Bronze Layer SSIS Packages*

Type	SSIS Package	Description
Hub	BRZ_HUB_Taxpayer.dtsx	Load taxpayer business keys (TaxID)
Hub	BRZ_HUB_Declaration.dtsx	Load declaration business keys (DeclarationID)
Hub	BRZ_HUB_Payment.dtsx	Load payment business keys (PaymentID)
Hub	BRZ_HUB_Officer.dtsx	Load officer business keys (OfficerCode)
Hub	BRZ_HUB_Owner.dtsx	Load owner business keys (OwnerID)
Hub	BRZ_HUB_Category.dtsx	Load category business keys (CategoryID)
Hub	BRZ_HUB_Structure.dtsx	Load structure business keys (StructureID)
Hub	BRZ_HUB_Activity.dtsx	Load activity business keys (ActivityID)
Hub	BRZ_HUB_AnnualDecl.dtsx	Load annual declaration business keys (AnnualDeclarationID)
Satellite	BRZ_SAT_Taxpayer.dtsx	Track taxpayer attribute changes
Satellite	BRZ_SAT_MonthlyDecl.dtsx	Track monthly declaration attribute changes
Satellite	BRZ_SAT_Payment.dtsx	Track payment attribute changes
Satellite	BRZ_SAT_Officer.dtsx	Track officer attribute changes
Satellite	BRZ_SAT_Owner.dtsx	Track owner attribute changes
Satellite	BRZ_SAT_Category.dtsx	Track category attribute changes

Satellite	BRZ_SAT_Structure.dtsx	Track structure attribute changes
Satellite	BRZ_SAT_Activity.dtsx	Track activity attribute changes
Satellite	BRZ_SAT_AnnualDecl.dtsx	Track annual declaration attribute changes
Link	BRZ_LNK_TaxpayerDeclaration.dtsx	Taxpayer-Declaration relationship
Link	BRZ_LNK_DeclarationPayment.dtsx	Declaration-Payment relationship
Link	BRZ_LNK_TaxpayerOfficer.dtsx	Taxpayer-Officer relationship
Link	BRZ_LNK_TaxpayerAnnualDecl.dtsx	Taxpayer-AnnualDeclaration relationship
Link	BRZ_LNK_TaxpayerOwner.dtsx	Taxpayer-Owner relationship

4.3 Silver Layer Implementation (Business Vault)

4.3.1 Goal

Transform raw vault data into business-enriched constructs that simplify querying and provide derived business metrics.

4.3.2 Point-in-Time Tables

Three PIT SSIS packages create daily snapshots by cross-applying the latest active Satellite record for each Hub at the specified snapshot date. This pre-computation eliminates the complex temporal joins that would otherwise be required to query satellite data as-of a specific date.

4.3.3 Bridge Table

The BRG_Taxpayer_Owner bridge pre-joins the Taxpayer-Owner link with current satellite data from both sides, providing a denormalized view of taxpayer-owner relationships with full attribute details.

4.3.4 Business Vault Tables

BUS_ComplianceScore calculates a composite compliance rating (0-100) for each taxpayer based on multiple factors: declaration submission timeliness, payment completion rates, penalty frequency, and historical compliance patterns. The scoring algorithm weights each factor to produce an actionable risk assessment.

BUS_MonthlyMetrics aggregates detailed monthly metrics per taxpayer including total revenue, total tax, payment amounts, declaration counts, and derived ratios. These pre-computed metrics serve as building blocks for Gold layer fact tables and direct analytical queries.

4.3.5 SSIS Packages Implemented

Table 4.3: *Silver Layer SSIS Packages*

Type	SSIS Package	Description
PIT	SLV_PIT_Taxpayer.dtsx	Snapshot of taxpayer attributes at a point in time
PIT	SLV_PIT_Declaration.dtsx	Snapshot of declaration attributes at a point in time
PIT	SLV_PIT_Payment.dtsx	Snapshot of payment attributes at a point in time
Bridge	SLV_BRG_Taxpayer_Owner.dtsx	Pre-joined taxpayer-owner relationship details
Business	SLV_BUS_ComplianceScore.dtsx	Taxpayer compliance scoring (0-100 scale)
Business	SLV_BUS_MonthlyMetrics.dtsx	Monthly aggregated metrics per taxpayer

4.4 Gold Layer Implementation (Information Vault)

4.4.1 Goal

Deliver a consumption-ready star schema information vault for business intelligence reporting and dashboard creation.

4.4.2 Dimension Tables

Seven dimension tables were implemented. DIM_Taxpayer and DIM_Officer use SCD Type 2 with effective date ranges, current record flags, and hash key tracking for change detection. DIM_Category, DIM_Structure, DIM_Activity, DIM_PaymentMethod, and DIM_Status use SCD Type 1, performing in-place updates when source attributes change.

4.4.3 Fact Tables

Four fact tables capture different analytical perspectives: FACT_MonthlyDeclaration stores declaration-level measures with dimension keys, FACT_Payment stores payment transactions, FACT_MonthlySnapshot provides pre-aggregated monthly summaries, and FACT_DeclarationLifecycle tracks the complete lifecycle of each declaration from submission to payment.

4.4.4 SSIS Packages Implemented

Table 4.4: *Gold Layer SSIS Packages*

Type	SSIS Package	SCD Type
Dimension	GLD_DIM_Taxpayer.dtsx	Type 2
Dimension	GLD_DIM_Officer.dtsx	Type 2
Dimension	GLD_DIM_Category.dtsx	Type 1
Dimension	GLD_DIM_PaymentMethod.dtsx	Type 1
Dimension	GLD_DIM_Activity.dtsx	Type 1
Dimension	GLD_DIM_Structure.dtsx	Type 1
Dimension	GLD_DIM_Status.dtsx	Type 1
Fact	GLD_FACT_MonthlyDeclaration.dtsx	-
Fact	GLD_FACT_Payment.dtsx	-
Fact	GLD_FACT_MonthlySnapshot.dtsx	-
Fact	GLD_FACT_DeclarationLifecycle.dtsx	-

4.5 Master Orchestration

4.5.1 Goal

Execute the complete ETL pipeline across all four layers in a single automated process with logging and error handling.

4.5.2 Implementation

The Master SSIS package Master_Complete_Pipeline.dtsx orchestrates the complete pipeline through 8 orchestrator packages that in turn call 49 child packages across the four layers. Each child package includes its own OnError Event Handler, allowing the pipeline to continue processing even if individual steps fail. The master package tracks total error count and logs the overall batch status (Success or Failed) based on whether any errors occurred.

4.5.3 Execution Flow

1. Start master batch (Execute SQL Task calls ETL_Control.usp_StartBatch)
2. Layer 1 - Staging: Load 9 staging tables
3. Layer 2 - Bronze: Load 9 Hubs, then 9 Satellites, then 5 Links
4. Layer 3 - Silver: Load 3 PITs, 1 Bridge, 2 Business tables
5. Layer 4 - Gold: Load 7 Dimensions, then 4 Facts
6. End master batch with summary statistics

4.5.4 SSIS Package Architecture

The SSIS implementation follows a hierarchical package design with three master packages: Master_Complete_Pipeline.dtsx (entry point), Master_Full_Load.dtsx, and Master_Incremental_Load.dtsx. The entry point package evaluates the load type and branches to the appropriate full or incremental pipeline. Each pipeline calls 8 orchestrator packages (2 Staging, 3 Bronze, 1 Silver, 2 Gold) which in turn call child packages for individual table loads. All packages implement OnError event handlers at both child and orchestrator levels: child packages call usp_EndStep + usp_LogError (6 parameters, 4 required) to mark failed steps, while orchestrators call usp_LogError only since they do not own steps. Master_Complete_Pipeline.dtsx owns the batch lifecycle and calls usp_EndBatch on error. Error propagation is controlled via Propagate=False and

MaximumErrorCount settings to allow sibling packages to continue when independent steps fail.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Results

5.1.1 Pipeline Execution Summary

Table 5.1: *Pipeline Execution Summary*

Layer	Objects Loaded	SSIS Packages	Status
Staging	9 tables	9 + 2 orchestrators	Success
Bronze (Data Vault)	9 Hubs, 9 Satellites, 5 Links	23 + 3 orchestrators	Success
Silver (Business Vault)	3 PITs, 1 Bridge, 2 Business	6 + 1 orchestrator	Success
Gold (Dimensional)	7 Dimensions, 4 Facts	11 + 2 orchestrators	Success

5.1.2 Data Volume Summary

Table 5.2: *Data Volume Summary*

Entity	Source Records	Description
Taxpayers	1,000	Registered taxpayers across 5 categories
Owners	1,956	1-3 owners per taxpayer
Officers	10	Tax collection officers
Monthly Declarations	38,491	4 years x 12 months x taxpayers
Annual Declarations	2,785	4 years x taxpayers at 90% rate
Payments	13,733	70-80% of approved declarations
Categories	5	Taxpayer classification lookup

Structures	5	Business structure lookup
Activities	5	Business activity lookup
Total	**57,990**	**Across 9 source tables (3 lookup, 3 reference, 3 transaction)**
Categories	5	Taxpayer classification lookup
Structures	5	Business structure lookup
Activities	5	Business activity lookup
Total	**57,990**	**Across 9 source tables (3 lookup, 3 reference, 3 transaction)**

5.1.3 Architecture Object Count

Table 5.3: *Architecture Object Count*

Component	Count
Source Tables	9
Staging Tables	9
Bronze Hubs	9
Bronze Satellites	9
Bronze Links	5
Silver PIT Tables	3
Silver Bridge Tables	1
Silver Business Tables	2
Gold Dimensions	7
Gold Fact Tables	4
ETL Control Tables (5 active, 4 framework)	9
SSIS Packages	60
Total Database Objects	67

5.2 Discussion

This section presents proof of concept evidence demonstrating how each challenge identified in the Problem Statement (Section 1.2) is addressed by the implemented Data Vault 2.0 methodology. The Data Vault 2.0 core (Hub-Satellite-Link modeling, insert-only loading, hash key generation) provides the fundamental solutions, while the medallion architecture organizes these solutions into clearly separated processing layers. For each challenge, the actual implementation is demonstrated with SQL code snippets, database object details, and simulated query results. The reproducible SQL scripts supporting these demonstrations are documented in the companion POC Implementation Guide [33] and Implementation Scenarios [32].

5.2.1 Challenge 1: Schema Rigidity

The Problem Statement identified that traditional star schema models require significant restructuring when source systems change [3,21]. In Cambodia's government tax systems, regulatory changes frequently introduce new data attributes, new tax types, or modified reporting requirements. Under a star schema approach, such changes require ALTER TABLE operations on dimension tables, ETL pipeline modifications, index rebuilds, and downstream report updates — affecting multiple objects simultaneously.

Data Vault 2.0 resolves this through its hub-satellite separation pattern [1,5]. Business keys are stored in Hubs (which rarely change), while descriptive attributes are stored in Satellites. When a new attribute is needed, a new Satellite table is simply created and linked to the existing Hub via the hash key. No existing objects are modified. The following figure demonstrates this capability:

Figure 5.1: POC — Schema Flexibility via Hub-Satellite Separation

Figure 5.1 — POC: Schema Flexibility via Hub-Satellite Separation

Scenario: A new attribute "TaxpayerEmail" is required — demonstrating how Data Vault 2.0 accommodates source changes without modifying existing objects

CURRENT BRONZE LAYER STRUCTURE (BEFORE CHANGE)

Object	Type	Key Columns	Purpose
HUB_Taxpayer	Hub	HUB_Taxpayer_HK, TaxID	Business key — never changes
SAT_Taxpayer	Satellite	HUB_Taxpayer_HK, SAT_LoadDate	Descriptive: Name, Revenue, Category
LNK_TaxpayerDeclaration	Link	HUB_Taxpayer_HK, HUB_Declaration_HK	Relationship — never changes
LNK_TaxpayerOfficer	Link	HUB_Taxpayer_HK, HUB_Officer_HK	Relationship — never changes

ADDING NEW ATTRIBUTE: CREATE NEW SATELLITE (ONLY ACTION REQUIRED)

```
-- New Satellite — no existing objects modified
CREATE TABLE DV_Bronze.dbo.SAT_Taxpayer_Contact (
    HUB_Taxpayer_HK   VARBINARY(32)  NOT NULL,
    SAT_LoadDate      DATETIME       NOT NULL,
    SAT_EndDate       DATETIME       NULL,
    SAT_RecordSource  VARCHAR(50)    NOT NULL,
    SAT_HashDiff      VARBINARY(32)  NOT NULL,
    TaxpayerEmail     VARCHAR(200),
    TaxpayerPhone     VARCHAR(50),
    CONSTRAINT PK_SAT_Taxpayer_Contact PRIMARY KEY (HUB_Taxpayer_HK, SAT_LoadDate),
    CONSTRAINT FK_SAT_TpContact_HUB FOREIGN KEY (HUB_Taxpayer_HK)
        REFERENCES DV_Bronze.dbo.HUB_Taxpayer (HUB_Taxpayer_HK)
);
```

AFTER CHANGE — IMPACT ASSESSMENT

Object	Type	Impact
HUB_Taxpayer	Hub	✓ No change — business key unchanged
SAT_Taxpayer	Satellite	✓ No change — existing attributes intact
LNK_TaxpayerDeclaration	Link	✓ No change — relationship unchanged
LNK_TaxpayerOfficer	Link	✓ No change — relationship unchanged
SAT_MonthlyDecl	Satellite	✓ No change — independent satellite
All SSIS Packages (60)	ETL	✓ No change — existing packages untouched
SAT_Taxpayer_Contact	**New Satellite**	**✓ Created independently, linked via HUB_Taxpayer_HK**

Result: New attribute added by creating 1 new Satellite table + 1 new SSIS package. Zero existing objects modified. The Hub-Satellite separation pattern [1][5] ensures that descriptive attribute changes never impact business keys or relationships.

As demonstrated, adding a new attribute such as TaxpayerEmail requires only creating a new SAT_Taxpayer_Contact Satellite, linked to the existing HUB_Taxpayer via the hash key. The impact assessment confirms zero modifications to existing Hubs, Satellites, Links, or SSIS packages (all 60 packages remain untouched). If the new attribute is needed in analytical outputs, only the Gold layer procedures that consume it require updating, while all other objects remain unchanged. This proves that the hub-satellite separation pattern successfully addresses the schema rigidity challenge.

5.2.2 Challenge 2: Historical Tracking

The second challenge identified that traditional SCD Type 1 approaches permanently overwrite previous values, losing historical data that government auditors require [3,21]. Tax administration mandates full audit trails showing exactly what data was recorded, when it changed, and what previous values were.

Data Vault 2.0 Satellites use an insert-only pattern where every change creates a new row with a LoadDateTime timestamp [1,5]. The HashDiff column (SHA-256 hash of descriptive attributes) enables efficient change detection — only genuinely changed records are inserted. This preserves a complete, immutable history of every data change.

Figure 5.2: POC — Full Historical Tracking via Insert-Only Satellite Pattern

Figure 5.2 — POC: Full Historical Tracking via Insert-Only Satellite Pattern

Scenario: Taxpayer TAX000001's EstimatedAnnualRevenue changes 3 times — demonstrating complete audit trail preservation

HASHDIFF-BASED CHANGE DETECTION (BRONZE LAYER STORED PROCEDURE)

```sql
-- From 06_ETL_Bronze_Procedures.sql: usp_Load_SAT_Taxpayer
-- Step 1: End-date existing current records where HashDiff changed
UPDATE sat SET sat.SAT_EndDate = GETDATE()
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN DV_Staging.dbo.STG_Taxpayer stg
    ON sat.HUB_Taxpayer_HK = stg.HUB_Taxpayer_HK
WHERE sat.SAT_EndDate IS NULL
  AND sat.SAT_HashDiff <> stg.SAT_HashDiff;   -- Only when attributes actually changed

-- Step 2: Insert new version for changed records
INSERT INTO DV_Bronze.dbo.SAT_Taxpayer (HUB_Taxpayer_HK, SAT_LoadDate, SAT_HashDiff, ...)
SELECT stg.HUB_Taxpayer_HK, GETDATE(), stg.SAT_HashDiff, ...
FROM DV_Staging.dbo.STG_Taxpayer stg
LEFT JOIN DV_Bronze.dbo.SAT_Taxpayer sat
    ON stg.HUB_Taxpayer_HK = sat.HUB_Taxpayer_HK AND sat.SAT_EndDate IS NULL
WHERE sat.HUB_Taxpayer_HK IS NULL                -- New record (no current version)
   OR sat.SAT_HashDiff <> stg.SAT_HashDiff;   -- Changed record (different hash)
```

QUERY RESULT — SAT_TAXPAYER: ALL 3 VERSIONS PRESERVED FOR TAX000001

HUB_Taxpayer_HK (first 8)	LegalBusinessName	Revenue	SAT_LoadDate	SAT_EndDate	Version
0x3A7F29B1...	Business 1	200,000	2026-02-22 08:00:15	2026-02-22 08:05:30	V1 (Historic)
0x3A7F29B1...	Business 1	350,000	2026-02-22 08:05:30	2026-02-22 08:10:45	V2 (Historic)
0x3A7F29B1...	Business 1 (AMENDED)	**505,000**	2026-02-22 08:10:45	*NULL*	**V3 (Current)**

Result: All 3 versions preserved with precise timestamps. SAT_EndDate = NULL identifies the current version. The insert-only pattern [1][5] with HASHBYTES('SHA2_256') change detection ensures only genuine changes are recorded — unchanged records are not duplicated. Government auditors can query any point-in-time using SAT_LoadDate.

The figure shows the actual Bronze layer stored procedure logic for HashDiff-based change detection using HASHBYTES('SHA2_256'). When a taxpayer's revenue changes three times (200,000 → 350,000 → 505,000), the insert-only Satellite preserves all three versions with precise timestamps. SAT_EndDate = NULL identifies the current version, while previous versions retain their exact load and end-date timestamps. This provides the complete regulatory audit trail required by tax authorities, enabling government auditors to query any point-in-time using SAT_LoadDate.

5.2.3 Challenge 3: Scalable ETL with Error Handling

The third challenge described how manual ETL scripts lack proper error handling, logging, and recovery mechanisms [19]. When failures occur partway through execution, there is no systematic way to determine what succeeded, what failed, or how to recover.

The implemented ETL Control Framework addresses this with a BatchLog and StepLog system [4,19]. Every SSIS child package includes OnError Event Handlers with automatic step-level logging. The framework records batch start/end times, step-level status (Success/Failed), row counts, execution duration, and detailed error messages. When failures occur, independent steps continue processing, and the failed step can be identified and re-executed without re-running the entire pipeline.

Figure 5.3: POC — ETL Control Framework with Step-Level Logging and Error Recovery

Figure 5.3 — POC: ETL Control Framework with Step-Level Logging and Error Recovery

Scenario: Load_STG_Payment fails with timeout during Staging layer execution — demonstrating error isolation and recovery capability

ETL_BATCHLOG — PIPELINE-LEVEL STATUS

BatchID	BatchType	BatchStatus	BatchStartTime	BatchEndTime	Duration
42	Full Load	⚠ **Partial**	2026-02-22 08:00:00	2026-02-22 08:02:01	121 sec

ETL_STEPLOG — STEP-LEVEL DETAIL (INDEPENDENT ERROR HANDLING PER PACKAGE)

StepLogID	StepName	StepStatus	RecordsProcessed	Duration (sec)	ErrorMessage
201	Load_STG_Category	✓ Success	5	1.2	
202	Load_STG_Structure	✓ Success	5	0.8	
203	Load_STG_Taxpayer	✓ Success	1,000	4.5	
204	**Load_STG_Payment**	✕ **Failed**	0	60.0	**Execution Timeout Expired [Source: TaxSystemDB.dbo.Payment]**
205	Load_STG_MonthlyDecl	✓ Success	38,487	18.3	
206	Load_STG_AnnualDecl	✓ Success	3,605	5.1	
207	Load_STG_Officer	✓ Success	10	0.3	
208	Load_STG_Owner	✓ Success	1,953	3.2	

ETL_ERRORLOG — DETAILED ERROR CAPTURE (ONERROR EVENT HANDLER)

ErrorLogID	BatchID	StepName	ErrorMessage	ErrorTime
15	42	Load_STG_Payment	Execution Timeout Expired. The timeout period elapsed prior to completion of the operation. [Source: TaxSystemDB.dbo.Payment, OLE DB provider]	2026-02-22 08:01:05

Result: The SSIS OnError Event Handler [4] captured the error via System::ErrorDescription and logged it to ETL_ErrorLog. Key outcomes: (1) Failed step identified instantly: Load_STG_Payment. (2) 7 of 8 steps completed successfully — independent error handling per package ensures failures don't cascade. (3) 45,065 rows processed across successful steps. (4) **Recovery: re-run only Load_STG_Payment** — no need to restart the entire pipeline.

The figure shows the actual ETL_BatchLog, ETL_StepLog, and ETL_ErrorLog output from the implemented framework. When Load_STG_Payment fails with a timeout error, the remaining steps (Load_STG_MonthlyDeclaration, Load_STG_AnnualDeclaration, Load_STG_Officer, Load_STG_Owner) continue processing successfully — 7 of 8 steps completed, processing 45,065 rows across successful steps. The SSIS OnError Event Handler captures the error via System::ErrorDescription and logs it to ETL_ErrorLog with the precise timestamp and source details. The administrator can immediately identify the failed step and re-execute only Load_STG_Payment without re-running the entire pipeline.

5.2.4 Challenge 4: Business Rules Gap

The fourth challenge identified a gap between raw operational data and business-ready analytical information. Tax administrators need derived metrics such as compliance scores, payment behavior patterns, and risk classifications that require complex business logic applied consistently.

The implemented architecture addresses this through two layers: the Silver (Business Vault) layer computes derived metrics such as compliance scores, on-time payment rates, and risk levels using consistent business rules [1,2]; the Gold (Star Schema) layer then organizes these metrics into dimensional models with pre-aggregated Fact tables suitable for BI dashboard consumption.

Figure 5.4: POC — Business Rules Applied via Silver (Business Vault) + Gold (Star Schema)

Figure 5.4 — POC: Business Rules Applied via Silver (Business Vault) + Gold (Star Schema)

Scenario: Derived analytics for TaxID TAX000001 — demonstrating pre-computed compliance metrics from raw Bronze data

SILVER LAYER: BUS_COMPLIANCESCORE (COMPUTED FROM BRONZE HUB + SATELLITE + LINK)

```sql
-- From 07_ETL_Silver_Procedures.sql: usp_Load_BUS_ComplianceScore
-- ComplianceScore = 50% FilingOnTimeRate + 50% PaymentOnTimeRate
SELECT hub.HUB_Taxpayer_HK,
       ISNULL(filing.OnTimeRate, 0) * 0.50 + ISNULL(payment.OnTimeRate, 0) * 0.50 AS ComplianceScore,
       filing.OnTimeRate  AS FilingOnTimeRate,
       payment.OnTimeRate AS PaymentOnTimeRate,
       penalties.PenaltyCount
FROM DV_Bronze.dbo.HUB_Taxpayer hub
OUTER APPLY (/* filing on-time calculation */) filing
OUTER APPLY (/* payment completion rate */) payment
OUTER APPLY (/* penalty count */) penalties;
```

72.5%	78.3%	66.7%	MEDIUM	2
Compliance Score	Filing On-Time Rate	Payment On-Time Rate	Risk Classification	Penalty Count

GOLD LAYER: FACT_MONTHLYDECLARATION (STAR SCHEMA — BI DASHBOARD READY)

DIM_Taxpayer	DIM_Status	Month	GrossRevenue	TaxAmount	TotalAmount	PaymentAmount	Coverage %
TAX000001	Approved	Jan 2023	45,000	6,750	6,750	6,750	100%
TAX000001	Approved	Feb 2023	52,000	7,800	7,800	7,800	100%
TAX000001	Rejected	Mar 2023	38,000	5,700	5,700	0	0%
TAX000001	Approved	Apr 2023	61,000	9,150	9,150	9,150	100%
TAX000001	Pending	May 2023	55,000	8,250	8,250	0	0%
TAX000001	Approved	Jun 2023	48,000	7,200	7,200	7,200	100%

Result: The Silver layer (Business Vault) computes derived metrics using consistent business rules [1][2] — compliance score formula: 50% filing on-time rate + 50% payment on-time rate. The Gold layer (Star Schema) organizes these into dimensional models with DIM/FACT joins for BI dashboard consumption. Tax administrators get instant, consistent analytics without writing ad-hoc queries across raw Bronze tables.

The figure demonstrates the actual Silver layer stored procedure (usp_Load_BUS_ComplianceScore) that computes derived metrics from Bronze Hub, Satellite, and Link tables. For taxpayer TAX000001, the Business Vault produces a compliance score of 72.5%, calculated as 50% filing on-time rate (78.3%) plus 50% payment on-time rate (66.7%), along with a risk classification of MEDIUM and penalty count of 2. The Gold layer FACT_MonthlyDeclaration table then organizes these into a star schema with DIM/FACT joins, showing period-by-period revenue, tax amounts, and payment coverage. These pre-computed metrics enable tax administrators to quickly identify high-risk taxpayers, monitor compliance trends, and make data-driven enforcement decisions without writing complex ad-hoc queries across raw Bronze tables.

5.2.4.1 Summary of POC Results

Table 5.4: *Summary of POC Results*

Challenge	Problem	DV 2.0 Solution	POC Result
1. Schema Rigidity	ALTER table impacts multiple objects	Create new Satellite only	Zero impact on existing objects

2. Historical Tracking	Overwrite patterns lose history	Insert-only Satellite with timestamps	All versions preserved with audit trail
3. Scalable ETL	No logging or error recovery	BatchLog + StepLog + OnError Event Handlers	Full visibility with step-level recovery
4. Business Rules Gap	Raw data with no derived analytics	Business Vault + Gold Star Schema	Pre-computed scores, trends, risk levels

5.2.4.2 What Worked Well

1. Data Vault Flexibility: The hub-satellite-link pattern successfully decoupled business keys from descriptive attributes [1,5]. Adding new attributes to existing entities required only modifying the relevant Satellite, without impacting Hubs, Links, or other Satellites.
2. Medallion Architecture: The four-layer separation provided clear boundaries for troubleshooting [2,26]. When data quality issues were identified in the Gold layer, tracing back through Silver and Bronze to the Staging layer was straightforward.
3. Hash Key Approach: Using SHA-256 hash keys [1,8] via HASHBYTES('SHA2_256') eliminated the need for key lookup tables during loading, simplifying the ETL logic and enabling deterministic key generation.
4. Incremental Loading: The watermark-based approach successfully loaded only changed records, significantly reducing processing time for subsequent runs after the initial full load.
5. Error Isolation: The per-package OnError Event Handler pattern in the SSIS orchestration ensured that failures in individual entities did not block processing of independent entities.

5.2.4.3 Challenges and Limitations

1. Hash Collision Risk: SHA-256 hash keys have an extremely low theoretical collision probability (1 in 2^128) [1,8], which is negligible for the data volumes in

this implementation and production tax systems alike. The choice of SHA-256 over MD5 provides stronger collision resistance suitable for government-grade data warehousing.

2. Simulated Data: The use of generated sample data means the system has not been validated against real-world data quality challenges such as missing values, encoding issues, and referential integrity violations in source data.
3. No Cross-Source Integration: The architecture was designed for a single source system. Multi-source integration would require additional Same-As Links and Multi-Active Satellites [1,6].
4. Performance at Scale: The system was not load-tested with production-scale data volumes. Performance optimizations such as partition switching, column store indexes, and parallel loading may be needed for larger datasets.

5.2.4.4 Best Practices Identified

1. Always implement ETL control frameworks with batch/step logging before building data loading SSIS packages [4,19].
2. Use hash keys consistently across all layers to maintain traceability from Gold back to Bronze [1,8].
3. Design staging SSIS packages to support both full and incremental loading from the start.
4. Implement master orchestration with individual error handling per step rather than failing the entire pipeline on the first error [4].
5. Separate database instances per layer to enable independent backup, restore, and security policies.

5.3 Threats to Validity

5.3.1 Internal Validity

The proof-of-concept demonstrations were conducted on a single development environment configuration (Google Cloud e2-standard-4 VM, 16GB RAM). Variations in hardware specifications, SQL Server configuration parameters, and concurrent workload conditions could affect the observed ETL execution behavior. To mitigate this, all POC

demonstrations were executed under identical conditions on the same hardware, and results were verified through reproducible SQL scripts.

5.3.2 External Validity

The implementation uses simulated tax data rather than real operational records from Cambodia's General Department of Taxation (GDT). While the data generator produces realistic distributions (1,000 taxpayers, 4 years of declarations) as verified in the companion Source Database Verification document [34], real-world data introduces additional complexities including missing values, encoding issues, data quality anomalies, and referential integrity violations that were not evaluated. The generalizability of the POC results to production-scale environments requires further validation.

5.3.3 Construct Validity

The evaluation criteria used in the POC assessment were selected based on the identified challenges in Section 1.2 and informed by prior comparative studies [9-11]. However, additional criteria such as data governance integration, real-time processing capability, quantitative performance benchmarking, and team skill requirements may also be relevant for a thorough technology selection decision in production contexts.

5.3.4 Conclusion Validity

The POC demonstrations provide qualitative evidence that Data Vault 2.0 addresses the four identified challenges. However, the evaluation is based on a single implementation with simulated data at proof-of-concept scale. Quantitative performance benchmarking (ETL load times, query execution performance, scalability analysis) is identified as future work to strengthen the validity of the conclusions.

CHAPTER 6

CONCLUSION AND FUTURE WORK

The study successfully designed, implemented, and evaluated a complete Data Vault 2.0 data warehouse for the Cambodia tax system using medallion architecture on Microsoft SQL Server 2025. The implementation demonstrates that Data Vault 2.0 provides a reliable, auditable, and flexible foundation for data warehousing in government tax administration contexts where historical completeness, source system adaptability, and regulatory compliance are critical requirements.

The Data Vault 2.0 methodology proved effective as the core modeling approach, with its Hub-Satellite-Link pattern enabling flexible schema evolution, complete historical tracking, and full audit traceability. The four-layer medallion architecture (Staging, Bronze, Silver, Gold) organized the Data Vault 2.0 implementation into clearly separated processing layers, aligning with Microsoft's recommended medallion lakehouse architecture for enterprise data platforms [26]. The Bronze layer's Data Vault model preserves the full history of source data changes, the Silver layer's Business Vault derives actionable metrics from raw vault data, and the Gold layer's star schema delivers consumption-ready dimensional models for business intelligence reporting. The ETL control framework provides the observability and reliability needed for production-grade data pipelines.

The complete implementation comprises 9 staging tables, 23 Bronze objects (9 Hubs, 9 Satellites, 5 Links), 6 Silver objects (3 PITs, 1 Bridge, 2 Business Vault tables), 11 Gold objects (7 Dimensions, 4 Facts), orchestrated through a master pipeline with full batch and step logging.

The proof-of-concept evaluation across four identified challenges — schema rigidity, historical tracking, scalable ETL, and business rules derivation — provides concrete evidence that the implemented architecture addresses each challenge through specific design patterns. Prior comparative studies [9-11] indicate that Data Vault 2.0 offers superior schema adaptability, historical completeness, and auditability compared to Kimball dimensional modeling, at the cost of moderate increases in ETL complexity and storage footprint. The POC evidence from this implementation is consistent with those

findings, and these trade-offs are justified in Cambodia's government tax administration context where regulatory compliance, auditability, and long-term adaptability outweigh raw performance considerations.

The research contributes to the limited body of peer-reviewed literature on Data Vault 2.0 implementations by providing a complete end-to-end implementation and domain-specific evidence from the Cambodia tax administration context.

Future Work:

1. Integrate Real Cambodian Tax Data: Replace simulated data with anonymized real tax records from the General Department of Taxation to validate the architecture against actual data quality challenges and business requirements.
2. Multi-Source Integration: Extend the architecture to integrate data from multiple source systems (e.g., customs, property tax, VAT) using Same-As Links and Multi-Active Satellites.
3. Real-Time Data Processing: Implement Change Data Capture (CDC) or event-driven architecture to support near-real-time data loading for time-sensitive compliance monitoring.
4. Advanced Analytics: Build machine learning models on top of the Gold layer for predictive analytics such as compliance risk scoring, revenue forecasting, fraud detection, and anomaly detection.
5. Performance Benchmarking and Optimization: Conduct quantitative performance benchmarks measuring ETL load times across all four layers (full load vs incremental load), query execution performance at multiple complexity levels comparing Gold layer vs Raw Vault queries, and scalability analysis with progressively increasing data volumes. Implement materialized views [27], column store indexes, table partitioning, and parallel loading strategies for production-scale data volumes.
6. Data Governance: Add row-level security, data masking, and role-based access control to protect sensitive taxpayer information.
7. Dashboard Integration: Connect the Gold layer to Power BI or similar BI tools to create interactive dashboards for tax administrators and policy makers.

REFERENCES

[1] D. Linstedt and M. Olschimke, "Building a Scalable Data Warehouse with Data Vault 2.0," Burlington, MA: Morgan Kaufmann, 2015.

[2] Databricks, "What is the Medallion Lakehouse Architecture?" Databricks Documentation, 2025. https://docs.databricks.com/aws/en/lakehouse/medallion

[3] R. Kimball and M. Ross, "The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling," 3rd ed., Indianapolis, IN: Wiley, 2013.

[4] Microsoft, "SQL Server Integration Services (SSIS)," Microsoft Documentation, 2024. https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services

[5] Scalefree / Data Vault Alliance, "Data Vault 2.0 Standards and Best Practices," Scalefree International, 2024. https://www.scalefree.com/consulting/data-vault-2-0/

[6] P. Hultgren, "Modeling the Agile Data Warehouse with Data Vault," Brighton, UK: New Hamilton, 2012.

[7] W. H. Inmon, "Building the Data Warehouse," 4th ed., Indianapolis, IN: Wiley, 2005.

[8] K. Graziano, "The Data Vault Guru: A Pragmatic Guide on Building a Data Vault," New York, NY: Apress, 2021.

[9] L. Yessad and A. Labiod, "Comparative Study of Data Warehouses Modeling Approaches: Inmon, Kimball and Data Vault," in Proc. 2016 Int. Conf. System Reliability and Science (ICSRS), Paris, France, pp. 95-99, 2016.

[10] Z. Naamane and V. Jovanovic, "Effectiveness of Data Vault compared to Dimensional Data Marts on Overall Performance of a Data Warehouse System," Int. J. Computer Science Issues, vol. 13, no. 1, pp. 1-16, 2016.

[11] A. Vines, "Performance Evaluation of Data Vault and Dimensional Modeling: Insights from TPC-DS Dataset Analysis," in Proc. 23rd Int. Conf. Informatics in Economy (IE 2024), Smart Innovation, Systems and Technologies, vol. 426, pp. 27-37, Springer, 2025.

[12] R. Singh, "Architectural Resilience in Enterprise Data Systems: A Comparative Analysis of Data Vault 2.0, Dimensional Modeling, and Hybrid Frameworks in Secure, High-Volume Environments," Int. J. Advance Scientific Research, 2025.

[13] Z. Bellahsene, "Schema Evolution in Data Warehouses," Knowledge and Information Systems, vol. 4, no. 3, pp. 283-304, Springer, 2002.

[14] M. Arora and A. Gosain, "Schema Evolution for Data Warehouse: A Survey," Int. J. Computer Applications, vol. 22, no. 6, pp. 6-14, 2011.

[15] D. Subotic, "Data Warehouse Schema Evolution Perspectives," in New Trends in Database and Information Systems II, Advances in Intelligent Systems and Computing, vol. 312, pp. 333-338, Springer, 2015.

[16] A. Nambiar and D. Mundra, "An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management," Big Data and Cognitive Computing, vol. 6, no. 4, 2022.

[17] A. Dhaouadi, K. Bousselmi, M. Gammoudi, S. Monnet, and F. Gargouri, "Data Warehousing Process Modeling from Classical Approaches to New Trends: Main Features and Comparisons," Data, vol. 7, no. 8, 2022.

[18] S. Ponnusamy, "Evolution of Enterprise Data Warehouse: Past Trends and Future Prospects," Int. J. Computer Trends and Technology (IJCTT), vol. 71, no. 9, pp. 1-6, 2023.

[19] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual Modeling for ETL Processes," in Proc. 5th ACM Int. Workshop on Data Warehousing and OLAP (DOLAP), pp. 14-21, ACM, 2002.

[20] M. Poess, B. Smith, L. Kollar, and P. A. Larson, "TPC-DS, Taking Decision Support Benchmarking to the Next Level," in Proc. ACM SIGMOD Int. Conf. Management of Data, pp. 582-587, 2002.

[21] M. Khalid, "Challenges of Dimensional Modeling in Business Intelligence Systems," Int. J. Computer & Organization Trends, vol. 5, no. 3, pp. 30-31, 2015.

[22] E. M. Leonard, "Design and Implementation of an Enterprise Data Warehouse," M.S. thesis, Marquette University, 2011.

[23] H. Fan and A. Poulovassilis, "Schema Evolution in Data Warehousing Environments — A Schema Transformation-Based Approach," in Conceptual Modeling — ER 2004, LNCS, vol. 3288, pp. 639-653, Springer, 2004.

[24] W. Oueslati and J. Akaichi, "A Survey on Data Warehouse Evolution," Int. J. Database Management Systems (IJDMS), vol. 2, no. 4, pp. 11-24, 2010.

[25] A. Vines and R. Samoila, "An Overview of Data Vault Methodology and Its Benefits," Informatica Economica, vol. 27, no. 1, pp. 99-110, 2023.

[26] Microsoft, "Implement Medallion Lakehouse Architecture in Microsoft Fabric," Microsoft Fabric Documentation, 2025. https://learn.microsoft.com/en-us/fabric/onelake/onelake-medallion-lakehouse-architecture

[27] Microsoft, "Create Indexed Views," Microsoft SQL Server Documentation, 2024. https://learn.microsoft.com/en-us/sql/relational-databases/views/create-indexed-views

[28] A. Gandomi and M. Haider, "Beyond the Hype: Big Data Concepts, Methods, and Analytics," International Journal of Information Management, vol. 35, no. 2, pp. 137–144, 2015.

[29] A. Oussous, F. Z. Benjelloun, A. A. Lahcen, and S. Belfkih, "Big Data Technologies: A Survey," Journal of King Saud University – Computer and Information Sciences, vol. 30, no. 4, pp. 431–448, 2018.

[30] S. Shukla, A. N. Nainwal, N. Joshi, R. A. Varma, and R. T. Irrinki, "Big Data Analytics for Tax Administration," in Big Data Analytics, BDA 2019, Lecture Notes in Computer Science, vol. 11932, pp. 47–65, Springer, 2019.

[31] Inter-American Center of Tax Administrations (CIAT), "Use of Big Data in Tax Administrations," CIAT, 2021. https://www.ciat.org/use-of-big-data-in-tax-administrations

[32] S. So, "Implementation Scenarios: Data Vault 2.0 Tax System Data Warehouse — Companion Document to Chapter 5," RUPP, 2026. [Step-by-step SQL scripts and SSMS screenshot evidence for Flexibility, Materialized Views, and Early Arriving Facts scenarios]

[33] S. So, "POC Implementation Guide: Data Vault 2.0 Tax System Data Warehouse — Companion Document to Chapter 5 (Sections 5.2.1–5.2.4)," RUPP, 2026. [Reproducible SQL scripts for POC demonstrations]

[34] S. So, “Source Database Verification: Data Vault 2.0 Tax System Data Warehouse — Companion Document to Chapter 4 (Section 4.1),” RUPP, 2026. [12-point validation checklist for simulated TaxSystemDB: 9 tables, 57,990 records, referential integrity, distributions]

APPENDICES

Appendix A: SQL Script Files

Table A.1: *SQL Script Files*

Script	Description	Lines
01_CreateDatabaseStructure.sql	Source database schema creation with tables, indexes, constraints	~280
02_TransactionData.sql	Sample data generation for all source tables	~300
03_ETL_Control_Setup.sql	ETL control framework (configuration, batch log, step log, watermark)	~1,015
04_DDL_Architecture_DW.sql	DDL for all 4 DW layers (Staging, Bronze, Silver, Gold tables)	~1,027
00_CleanAll_FreshFullLoad.sql	Truncate all data across 5 databases for fresh full-load reset	~160
10_Verify_FullLoad.sql	Verification queries for full load results across all layers	~500
11_Verify_IncrementalLoad.sql	Verification queries for incremental load change detection	~450
12_IncrementalTest_SourceChanges.sql	Insert/update source records to test incremental pipeline	~200

13_POC_Demonstrations.sql	POC evaluation scripts for all four challenges (Ch. 5)	~1,100
Source_Database_Verification.sql	12-point source database validation (companion to [34])	~320

Appendix B: SSIS Package Inventory

Table B.1: *SSIS Package Inventory*

Package	Type	Description
Master Packages (3)		
Master_Complete_Pipeline.dtsx	Master	Entry point; evaluates load type and branches
Master_Full_Load.dtsx	Master	Full load pipeline (calls 8 orchestrator packages)
Master_Incremental_Load.dtsx	Master	Incremental load pipeline (calls 8 orchestrator packages)
Orchestrator Packages (8)		
STG_Full_Load_All.dtsx	Orchestrator	Calls 9 staging packages with LoadType=FULL
STG_Incremental_Load_All.dtsx	Orchestrator	Calls 9 staging packages with LoadType=INCREMENTAL
BRZ_Load_All_Hubs.dtsx	Orchestrator	Loads all 9 Hub tables from staging
BRZ_Load_All_Links.dtsx	Orchestrator	Loads all 5 Link tables from staging
BRZ_Load_All_Satellites.dtsx	Orchestrator	Loads all 9 Satellite tables from staging

SLV_Load_All.dtsx	Orchestrator	Loads all Silver layer objects (PIT, Bridge, Business)
GLD_Load_All_Dimensions.dtsx	Orchestrator	Loads all 7 dimension tables
GLD_Load_All_Facts.dtsx	Orchestrator	Loads all 4 fact tables
Staging Layer Child Packages (9)		
STG_Taxpayer.dtsx	Child	Loads Taxpayer table to staging
STG_Owner.dtsx	Child	Loads Owner table to staging
STG_Officer.dtsx	Child	Loads Officer table to staging
STG_Category.dtsx	Child	Loads Category table to staging
STG_Structure.dtsx	Child	Loads Structure table to staging
STG_Activity.dtsx	Child	Loads Activity table to staging
STG_MonthlyDeclaration.dtsx	Child	Loads MonthlyDeclaration to staging
STG_AnnualDeclaration.dtsx	Child	Loads AnnualDeclaration to staging
STG_Payment.dtsx	Child	Loads Payment table to staging
Bronze Layer - Hub Packages (9)		
BRZ_HUB_Taxpayer.dtsx	Child	Loads HUB_Taxpayer from STG_Taxpayer
BRZ_HUB_Owner.dtsx	Child	Loads HUB_Owner from STG_Owner

BRZ_HUB_Officer.dtsx	Child	Loads HUB_Officer from STG_Officer
BRZ_HUB_Category.dtsx	Child	Loads HUB_Category from STG_Category
BRZ_HUB_Structure.dtsx	Child	Loads HUB_Structure from STG_Structure
BRZ_HUB_Activity.dtsx	Child	Loads HUB_Activity from STG_Activity
BRZ_HUB_Declaration.dtsx	Child	Loads HUB_Declaration from declarations
BRZ_HUB_AnnualDecl.dtsx	Child	Loads HUB_AnnualDecl from STG_AnnualDeclaration
BRZ_HUB_Payment.dtsx	Child	Loads HUB_Payment from STG_Payment
Bronze Layer - Link Packages (5)		
BRZ_LNK_TaxpayerDeclaration.dtsx	Child	Loads LNK_TaxpayerDeclaration
BRZ_LNK_TaxpayerAnnualDecl.dtsx	Child	Loads LNK_TaxpayerAnnualDecl
BRZ_LNK_TaxpayerOwner.dtsx	Child	Loads LNK_TaxpayerOwner
BRZ_LNK_TaxpayerOfficer.dtsx	Child	Loads LNK_TaxpayerOfficer
BRZ_LNK_DeclarationPayment.dtsx	Child	Loads LNK_DeclarationPayment
Bronze Layer - Satellite Packages (9)		
BRZ_SAT_Taxpayer.dtsx	Child	Loads SAT_Taxpayer with hash-based CDC

BRZ_SAT_Owner.dtsx	Child	Loads SAT_Owner with hash-based CDC
BRZ_SAT_Officer.dtsx	Child	Loads SAT_Officer with hash-based CDC
BRZ_SAT_Category.dtsx	Child	Loads SAT_Category with hash-based CDC
BRZ_SAT_Structure.dtsx	Child	Loads SAT_Structure with hash-based CDC
BRZ_SAT_Activity.dtsx	Child	Loads SAT_Activity with hash-based CDC
BRZ_SAT_MonthlyDecl.dtsx	Child	Loads SAT_MonthlyDecl with hash-based CDC
BRZ_SAT_AnnualDecl.dtsx	Child	Loads SAT_AnnualDecl with hash-based CDC
BRZ_SAT_Payment.dtsx	Child	Loads SAT_Payment with hash-based CDC
Silver Layer Child Packages (6)		
SLV_PIT_Taxpayer.dtsx	Child	Builds PIT_Taxpayer point-in-time table
SLV_PIT_Declaration.dtsx	Child	Builds PIT_Declaration point-in-time table
SLV_PIT_Payment.dtsx	Child	Builds PIT_Payment point-in-time table
SLV_BRG_Taxpayer_Owner.dtsx	Child	Builds Bridge_Taxpayer_Owner
SLV_BUS_ComplianceScore.dtsx	Child	Calculates compliance scores
SLV_BUS_MonthlyMetrics.dtsx	Child	Calculates monthly metrics
Gold Layer - Dimension Packages (7)		

GLD_DIM_Taxpayer.dtsx	Child	Loads DIM_Taxpayer dimension
GLD_DIM_Category.dtsx	Child	Loads DIM_Category dimension
GLD_DIM_Structure.dtsx	Child	Loads DIM_Structure dimension
GLD_DIM_Activity.dtsx	Child	Loads DIM_Activity dimension
GLD_DIM_Officer.dtsx	Child	Loads DIM_Officer dimension
GLD_DIM_Status.dtsx	Child	Loads DIM_Status dimension
GLD_DIM_PaymentMethod.dtsx	Child	Loads DIM_PaymentMethod dimension
Gold Layer - Fact Packages (4)		
GLD_FACT_MonthlyDeclaration.dtsx	Child	Loads FACT_MonthlyDeclaration
GLD_FACT_Payment.dtsx	Child	Loads FACT_Payment
GLD_FACT_DeclarationLifecycle.dtsx	Child	Loads FACT_DeclarationLifecycle
GLD_FACT_MonthlySnapshot.dtsx	Child	Loads FACT_MonthlySnapshot
Master_Complete_Pipeline.dtsx	Master	Entry point; evaluates load type and branches
Master_Full_Load.dtsx	Master	Full load pipeline (calls 8 orchestrator packages)
Master_Incremental_Load.dtsx	Master	Incremental load pipeline (calls 8 orchestrator packages)

Orchestrator Packages (8)		
STG_Full_Load_All.dtsx	Orchestrator	Calls 9 staging packages with LoadType=FULL
STG_Incremental_Load_All.dtsx	Orchestrator	Calls 9 staging packages with LoadType=INCREMENTAL
BRZ_Load_All_Hubs.dtsx	Orchestrator	Loads all 9 Hub tables from staging
BRZ_Load_All_Links.dtsx	Orchestrator	Loads all 5 Link tables from staging
BRZ_Load_All_Satellites.dtsx	Orchestrator	Loads all 9 Satellite tables from staging
SLV_Load_All.dtsx	Orchestrator	Loads all Silver layer objects (PIT, Bridge, Business)
GLD_Load_All_Dimensions.dtsx	Orchestrator	Loads all 7 dimension tables
GLD_Load_All_Facts.dtsx	Orchestrator	Loads all 4 fact tables
Staging Layer Child Packages (9)		
STG_Taxpayer.dtsx	Child	Loads Taxpayer table to staging
STG_Owner.dtsx	Child	Loads Owner table to staging
STG_Officer.dtsx	Child	Loads Officer table to staging
STG_Category.dtsx	Child	Loads Category table to staging
STG_Structure.dtsx	Child	Loads Structure table to staging
STG_Activity.dtsx	Child	Loads Activity table to staging

STG_MonthlyDeclaration.dtsx	Child	Loads MonthlyDeclaration to staging
STG_AnnualDeclaration.dtsx	Child	Loads AnnualDeclaration to staging
STG_Payment.dtsx	Child	Loads Payment table to staging
Bronze Layer - Hub Packages (9)		
BRZ_HUB_Taxpayer.dtsx	Child	Loads HUB_Taxpayer from STG_Taxpayer
BRZ_HUB_Owner.dtsx	Child	Loads HUB_Owner from STG_Owner
BRZ_HUB_Officer.dtsx	Child	Loads HUB_Officer from STG_Officer
BRZ_HUB_Category.dtsx	Child	Loads HUB_Category from STG_Category
BRZ_HUB_Structure.dtsx	Child	Loads HUB_Structure from STG_Structure
BRZ_HUB_Activity.dtsx	Child	Loads HUB_Activity from STG_Activity
BRZ_HUB_Declaration.dtsx	Child	Loads HUB_Declaration from declarations
BRZ_HUB_AnnualDecl.dtsx	Child	Loads HUB_AnnualDecl from STG_AnnualDeclaration
BRZ_HUB_Payment.dtsx	Child	Loads HUB_Payment from STG_Payment
Bronze Layer - Link Packages (5)		
BRZ_LNK_TaxpayerDeclaration.dtsx	Child	Loads LNK_TaxpayerDeclaration

BRZ_LNK_TaxpayerAnnualDecl.dtsx	Child	Loads LNK_TaxpayerAnnualDecl
BRZ_LNK_TaxpayerOwner.dtsx	Child	Loads LNK_TaxpayerOwner
BRZ_LNK_TaxpayerOfficer.dtsx	Child	Loads LNK_TaxpayerOfficer
BRZ_LNK_DeclarationPayment.dtsx	Child	Loads LNK_DeclarationPayment
Bronze Layer - Satellite Packages (9)		
BRZ_SAT_Taxpayer.dtsx	Child	Loads SAT_Taxpayer with hash-based CDC
BRZ_SAT_Owner.dtsx	Child	Loads SAT_Owner with hash-based CDC
BRZ_SAT_Officer.dtsx	Child	Loads SAT_Officer with hash-based CDC
BRZ_SAT_Category.dtsx	Child	Loads SAT_Category with hash-based CDC
BRZ_SAT_Structure.dtsx	Child	Loads SAT_Structure with hash-based CDC
BRZ_SAT_Activity.dtsx	Child	Loads SAT_Activity with hash-based CDC
BRZ_SAT_MonthlyDecl.dtsx	Child	Loads SAT_MonthlyDecl with hash-based CDC
BRZ_SAT_AnnualDecl.dtsx	Child	Loads SAT_AnnualDecl with hash-based CDC
BRZ_SAT_Payment.dtsx	Child	Loads SAT_Payment with hash-based CDC
Silver Layer Child Packages (6)		

SLV_PIT_Taxpayer.dtsx	Child	Builds PIT_Taxpayer point-in-time table
SLV_PIT_Declaration.dtsx	Child	Builds PIT_Declaration point-in-time table
SLV_PIT_Payment.dtsx	Child	Builds PIT_Payment point-in-time table
SLV_BRG_Taxpayer_Owner.dtsx	Child	Builds Bridge_Taxpayer_Owner
SLV_BUS_ComplianceScore.dtsx	Child	Calculates compliance scores
SLV_BUS_MonthlyMetrics.dtsx	Child	Calculates monthly metrics
Gold Layer - Dimension Packages (7)		
GLD_DIM_Taxpayer.dtsx	Child	Loads DIM_Taxpayer dimension
GLD_DIM_Category.dtsx	Child	Loads DIM_Category dimension
GLD_DIM_Structure.dtsx	Child	Loads DIM_Structure dimension
GLD_DIM_Activity.dtsx	Child	Loads DIM_Activity dimension
GLD_DIM_Officer.dtsx	Child	Loads DIM_Officer dimension
GLD_DIM_Status.dtsx	Child	Loads DIM_Status dimension
GLD_DIM_PaymentMethod.dtsx	Child	Loads DIM_PaymentMethod dimension
Gold Layer - Fact Packages (4)		
GLD_FACT_MonthlyDeclaration.dtsx	Child	Loads FACT_MonthlyDeclaration

GLD_FACT_Payment.dtsx	Child	Loads FACT_Payment
GLD_FACT_DeclarationLifecycle.dtsx	Child	Loads FACT_DeclarationLifecycle
GLD_FACT_MonthlySnapshot.dtsx	Child	Loads FACT_MonthlySnapshot